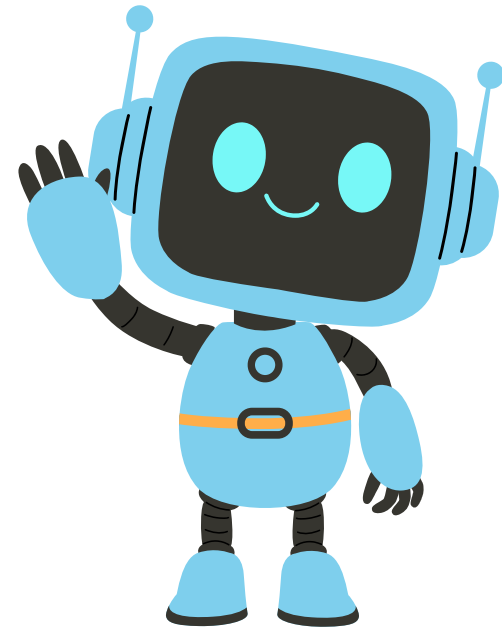




Language
Understanding
 $\lambda(\mu)$ Laboratory



LLM အလွန် ...

Architecture၊ Reasoning နှင့်

ခေတ်ထလာသော Agentic AI

(Overview)



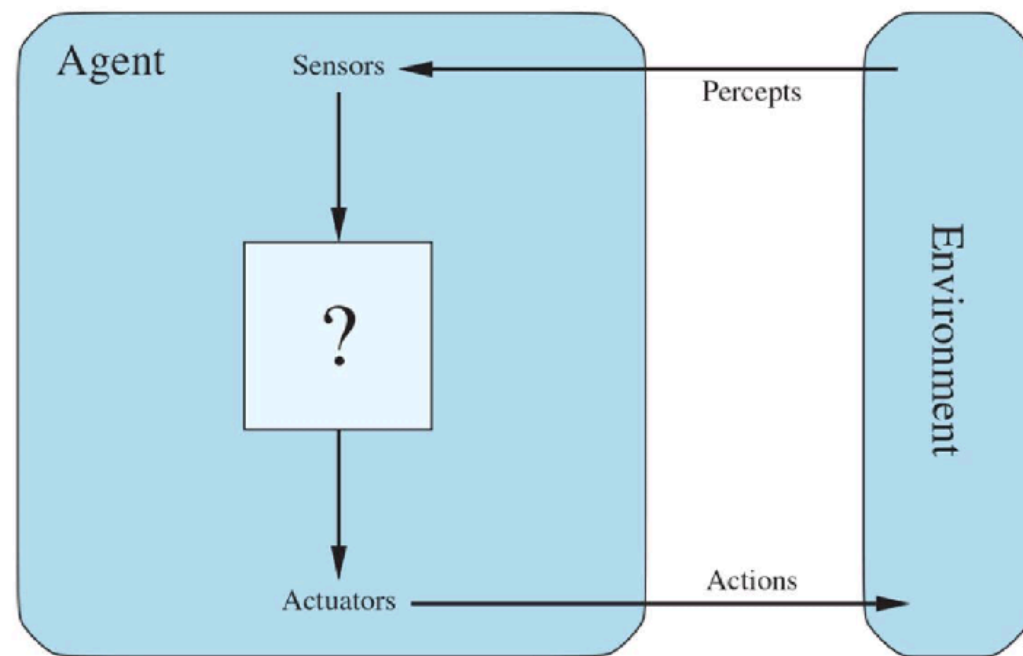
Thura Aung

SE15 @ KMITL

Core Member @ LU. Lab. Myanmar

thuraaungjune@github.io

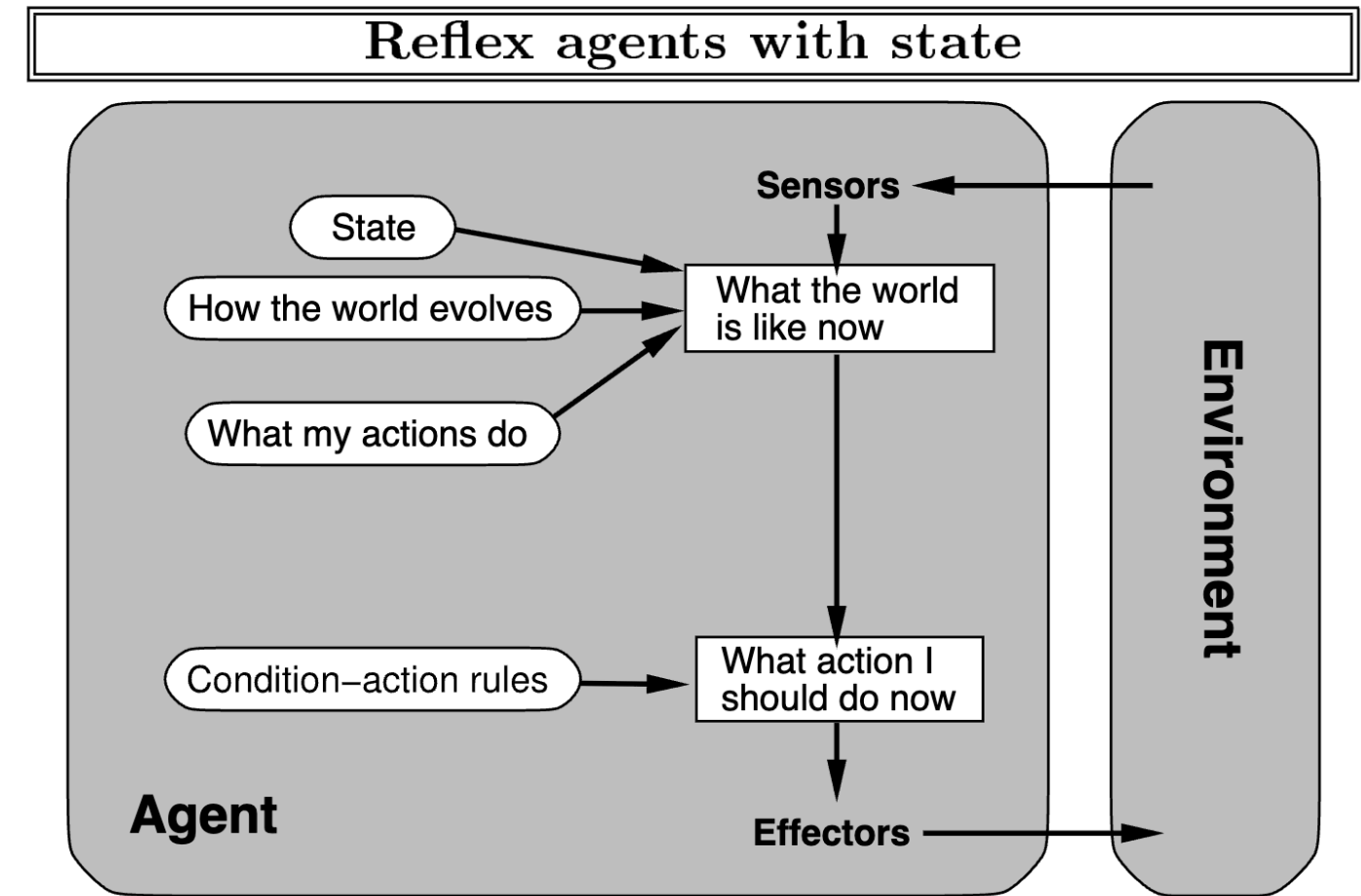
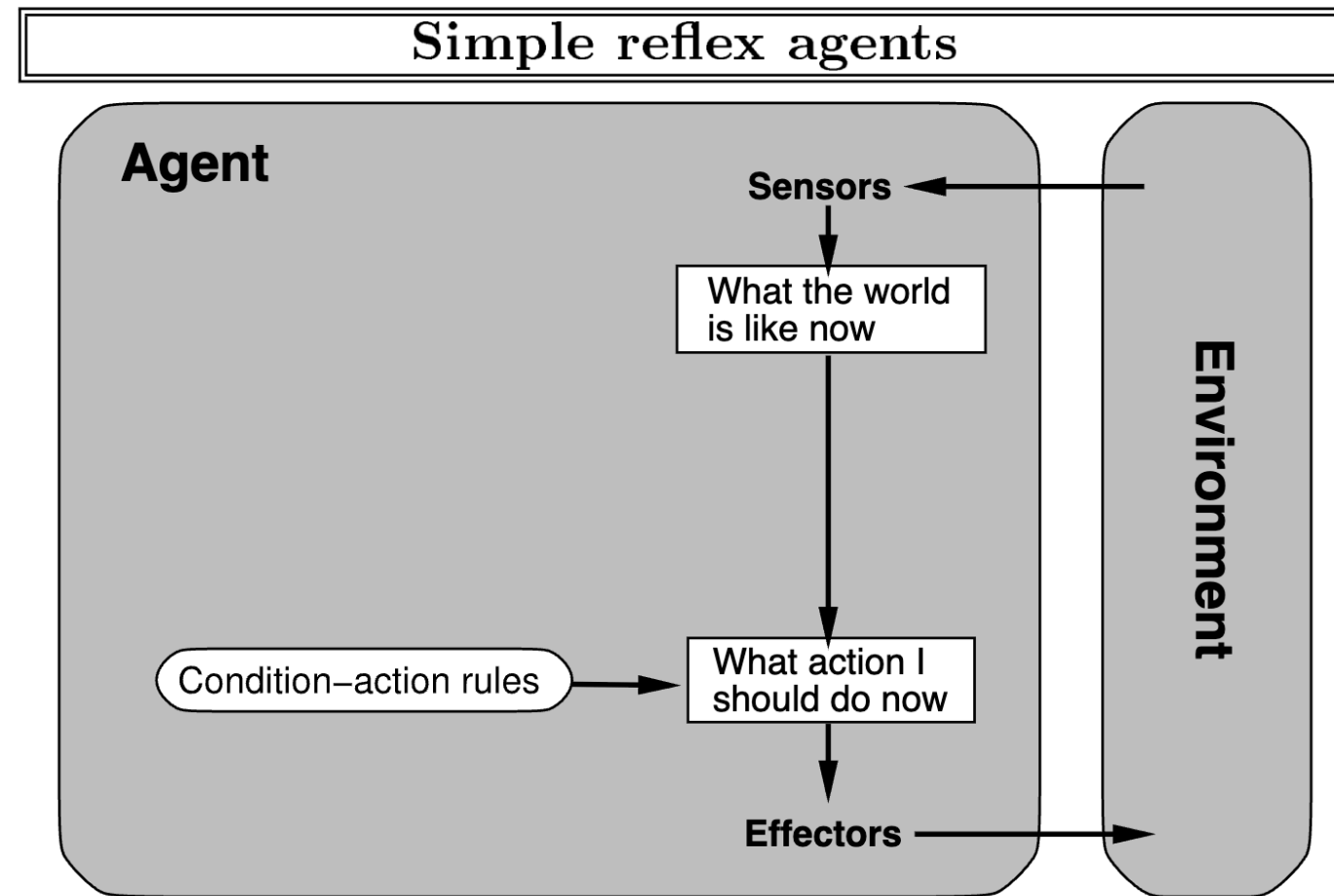
Agent ဆိုတာ ဘာလဲ။



Agent: Environment တစ်ခုခုနဲ့ ချိတ်ဆက်လုပ်ဆောင်ပေးတဲ့ intelligent စနစ်တစ်ခု

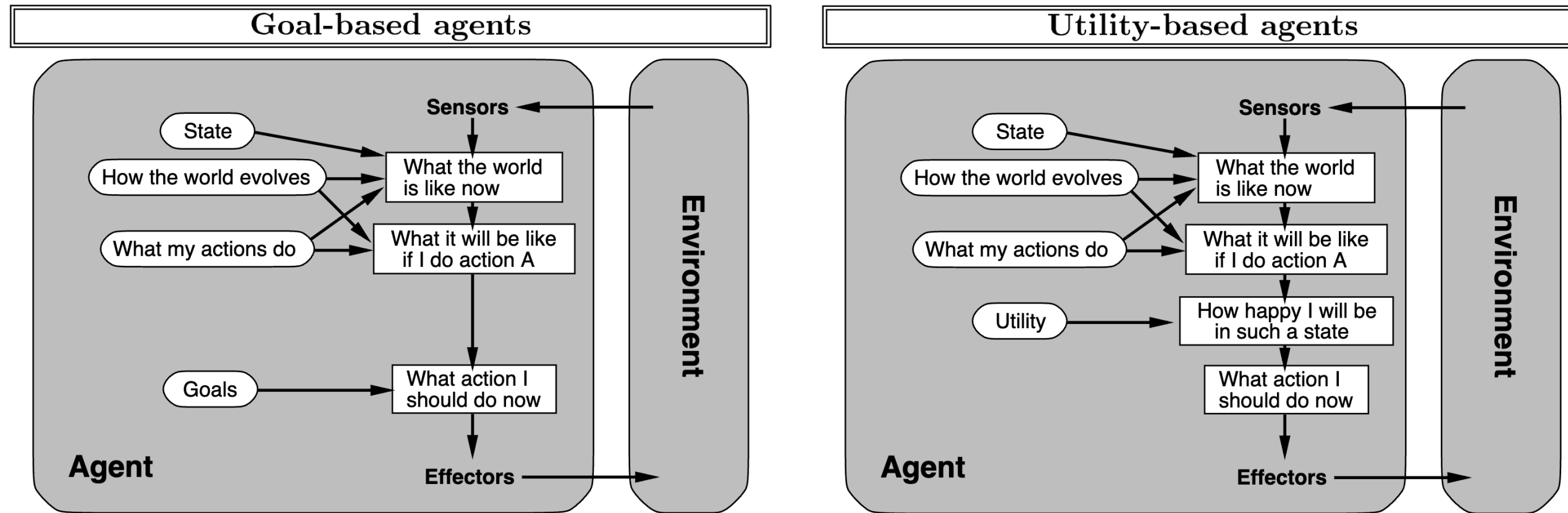
- Environments → (1) Physical (2) Digital environments
- ၁။ အာရုံခံခြင်း (Perceive): ပတ်ဝန်းကျင် အခြေအနေကို အာရုံခံကိရိယာများ (Sensors) (ဥပမာ- ကင်မရာ၊ မိုက်ခရိုဖုန်း၊ ကီးဘုတ်) မှတစ်ဆင့် သိရှိခြင်း။
- ၂။ ဆုံးဖြတ်ခြင်း (Reasoning & Decision): သိရှိလာသည့် အချက်အလက်များအပေါ် အခြေခံ၍ ကိုယ်ပိုင်ဗဟုသုတ (Knowledge base) သို့မဟုတ် စည်းမျဉ်းများ (Rules) ကို အသုံးပြုကာ မည်သည့်လုပ်ဆောင်ချက် လုပ်ရမည်ကို စဉ်းစားဆုံးဖြတ်ခြင်း။
- ၃။ လုပ်ဆောင်ခြင်း (Action): ဆုံးဖြတ်ချက်အတိုင်း စက်ကိရိယာ (Actuators) များ (ဥပမာ- စက်ရုပ်လက်တံ၊ စာသားရေးသားခြင်း၊ အသံထွက်ခြင်း) ကို အသုံးပြု၍ လက်တွေ့လှုပ်ရှားဆောင်ရွက်ခြင်း။

Agent ဆိုတာ ဘာလဲ။ (Cont'd)



- **Simple Reflex Agents:** အခြေအနေတစ်ခု ဖြစ်လာလျှင် ယခင်ချမှတ်ထားသော စည်းမျဉ်း (Rule) အတိုင်း ချက်ချင်း တုံ့ပြန်သည့် Agent ဖြစ်သည် (ဥပမာ - မီးပွိုင့်နီလျှင် ချက်ချင်းရပ်သည့် ကားစနစ်)
- **Model-Based Reflex Agents:** ယခုပတ်ဝန်းကျင် အခြေအနေအပြင်၊ ယခင်ဖြစ်ပျက်ခဲ့သော အခြေအနေများကိုပါ မှတ်သားထားပြီး ပိုမိုကောင်းမွန်စွာ တုံ့ပြန်နိုင်သော Agent ဖြစ်သည်

Agent ဆိုတာ ဘာလဲ။ (Cont'd)



- Goal-Based Agents: မိမိရောက်ရှိလိုသော ရည်မှန်းချက်တစ်ခုကို သတ်မှတ်ပေးထားပြီး ထိုရည်မှန်းချက် ပြည့်မီရန်အတွက် မည်သည့်လမ်းကြောင်းနှင့် နည်းလမ်းက အကောင်းဆုံးဖြစ်မည်ကို စီစဉ် (Plan) ပြီးမှ လှုပ်ရှားသော Agent ဖြစ်သည်။
- Utility-Based Agents: Goal-Based Agent နှင့် ဆင်တူသော်လည်း၊ အလုပ်တစ်ခု လုပ်ဆောင်ရာတွင် စွမ်းအင်အနည်းဆုံး သို့မဟုတ် အမြတ်အစွန်းအများဆုံးရရှိမည့် အကောင်းဆုံးနည်းလမ်း (Maximum Utility) ကို ရွေးချယ်တတ်သော Agent ဖြစ်သည်
- Learning Agents: မိမိ၏ ယခင်အမှား/အတွေ့အကြုံများမှတစ်ဆင့် သင်ယူလေ့လာပြီး နောက်တစ်ကြိမ်တွင် ပိုမိုထက်မြက်အောင် အလိုအလျောက် တိုးတက်ပြောင်းလဲနိုင်သော Agent ဖြစ်သည်

Language agent ဆိုတာ ဘာလဲ။

```
Welcome to

EEEEEE LL      IIII ZZZZZZ AAAAA
EE      LL      II      ZZ  AA  AA
EEEEEE LL      II      ZZ  AAAAAA
EE      LL      II      ZZ  AA  AA
EEEEEE LLLLLL IIII ZZZZZZ AA  AA

Eliza is a mock Rogerian psychotherapist.
The original program was described by Joseph Weizenbaum in 1966.
This implementation by Norbert Landsteiner 2005.

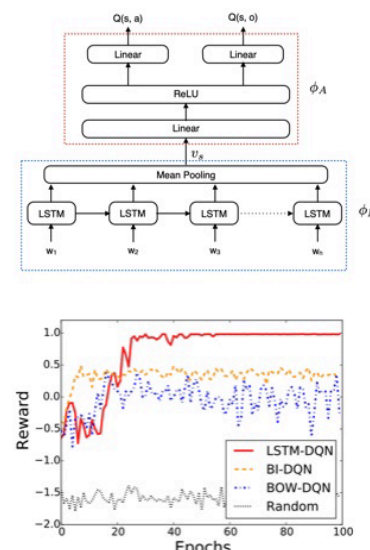
ELIZA: Is something troubling you ?
YOU:   Men are all alike.
ELIZA: What is the connection, do you suppose ?
YOU:   They're always bugging us about something or other.
ELIZA: Can you think of a specific example ?
YOU:   Well, my boyfriend made me come here.
ELIZA: Is it important to you that your boyfriend made you come here ?
YOU:   He says I'm depressed much of the time.
ELIZA: I am sorry to hear that you are depressed.
YOU:   It's true. I am unhappy.
ELIZA: Can you explain what made you unhappy ?
YOU:
```

ELIZA (1966)

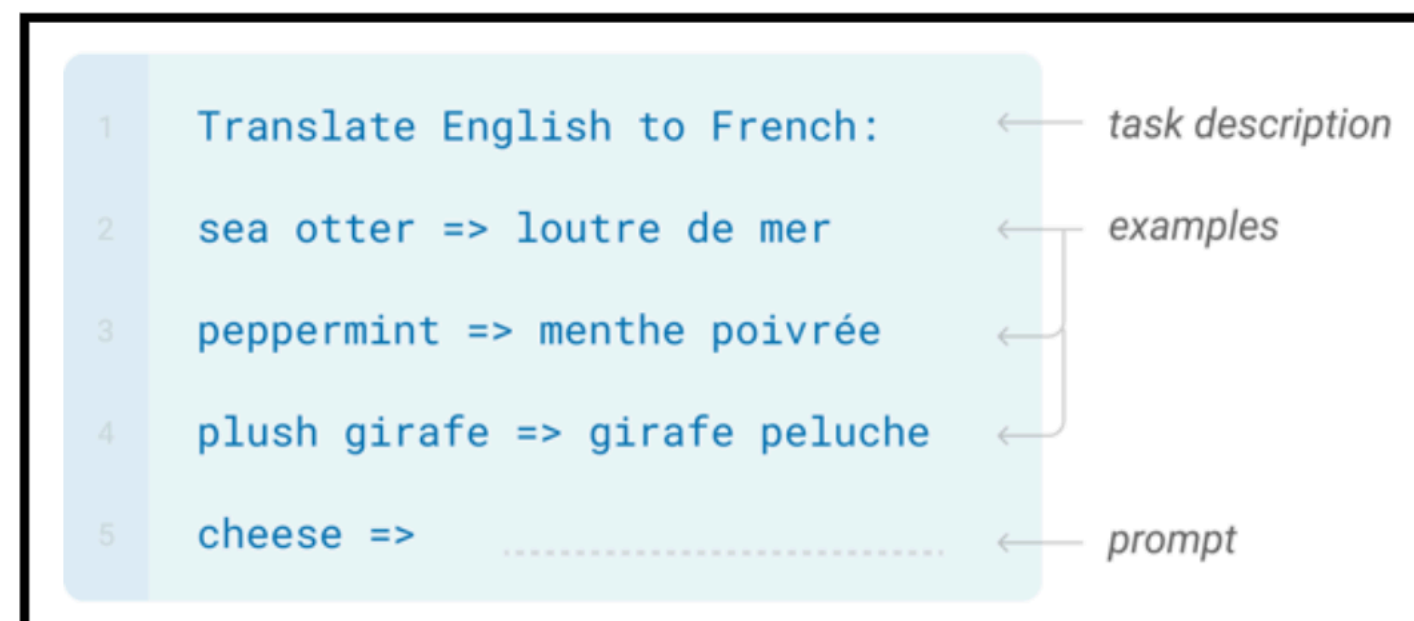
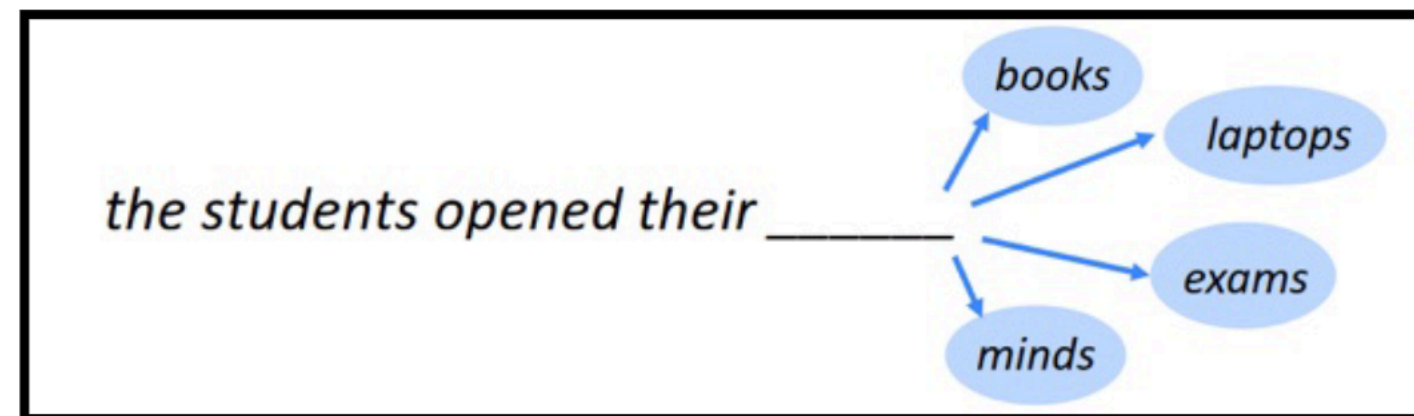
State 1: The old bridge
 You are standing very close to the bridge's eastern foundation. If you go east you will be back on solid ground ... The bridge sways in the wind.

Command: Go east

State 2: Ruined gatehouse
 The old gatehouse is near collapse. Part of its northern wall has already fallen down ... East of the gatehouse leads out to a small open area surrounded by the remains of the castle. There is also a standing archway offering passage to a path along the old southern inner wall.
 Exits: Standing archway, castle corner, Bridge over the abyss



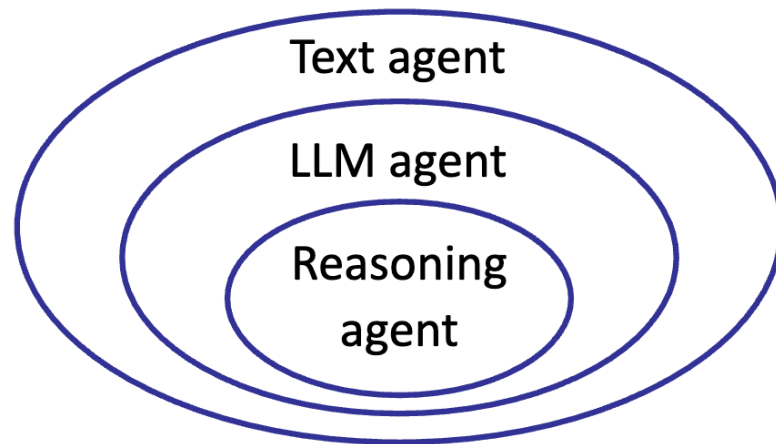
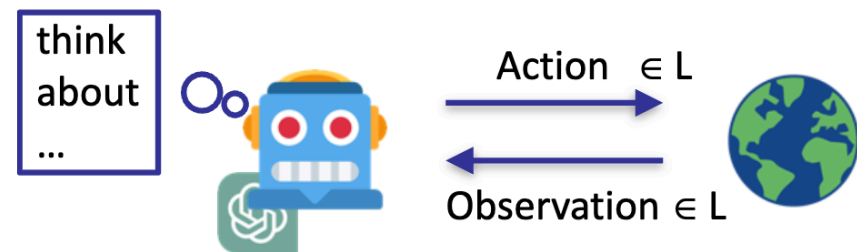
LSTM-DQN (2015)



Language Models are Few-Shot Learners NeurIPS 2020.

- Training: massive text corpora ပေါ် next-token prediction ပြုလုပ်ရန် train ခြင်း
- Inference: အမျိုးမျိုးသော task များအတွက် (few-shot) prompting ပြုလုပ်ခြင်း 5

Language agent ဆိုတာ ဘာလဲ။ (Cont'd)



GPT-4o

In this image, we see a young boy sitting on the floor, looking down at a pile of broken dishes. He appears to be sad or remorseful. In front of him, a woman, likely his mother or caregiver, is kneeling and looking at him with a concerned, questioning expression. Her body language, with her hands open and her facial expression, suggests that she's asking him something or expressing surprise or frustration about the broken dishes.

Perception

Intuitive Inference

Reasoning

- Level 1: Language agent

- Text action နှင့် observation ကို အသုံးပြုသည်။
- Examples: ELIZA, LSTM-DQN

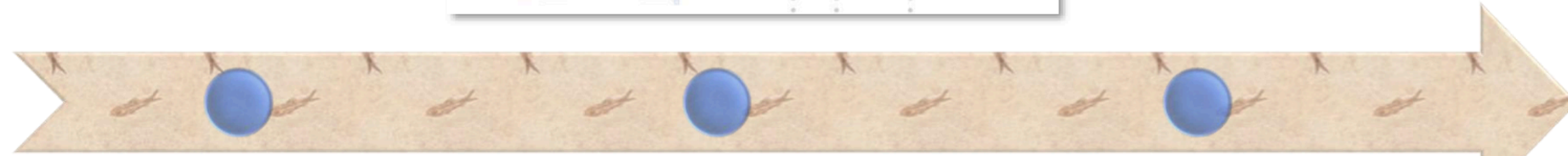
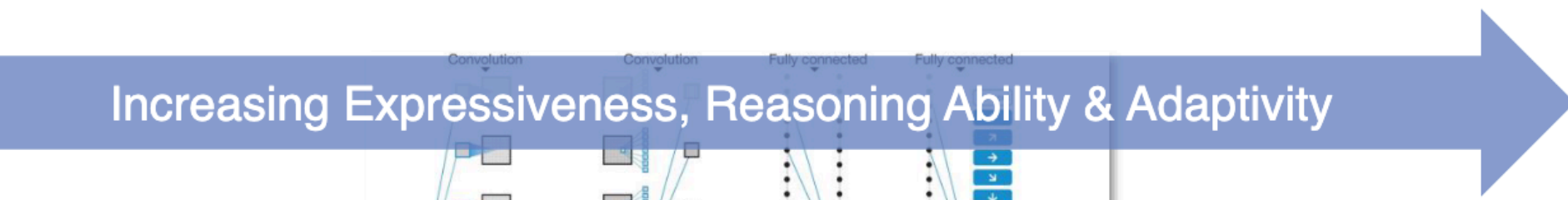
- Level 2: LLM agent

- Act လုပ်ရန်အတွက် LLM ကို အသုံးပြုသည်။
- Examples: SayCan, Language Planner

Level 3: Reasoning agent

- Act လုပ်ရန် စဉ်းစားတွေးခေါ်မှု (reason) အပိုင်းအတွက် LLM ကို အသုံးပြုသည်။
- Examples: ReAct, AutoGPT

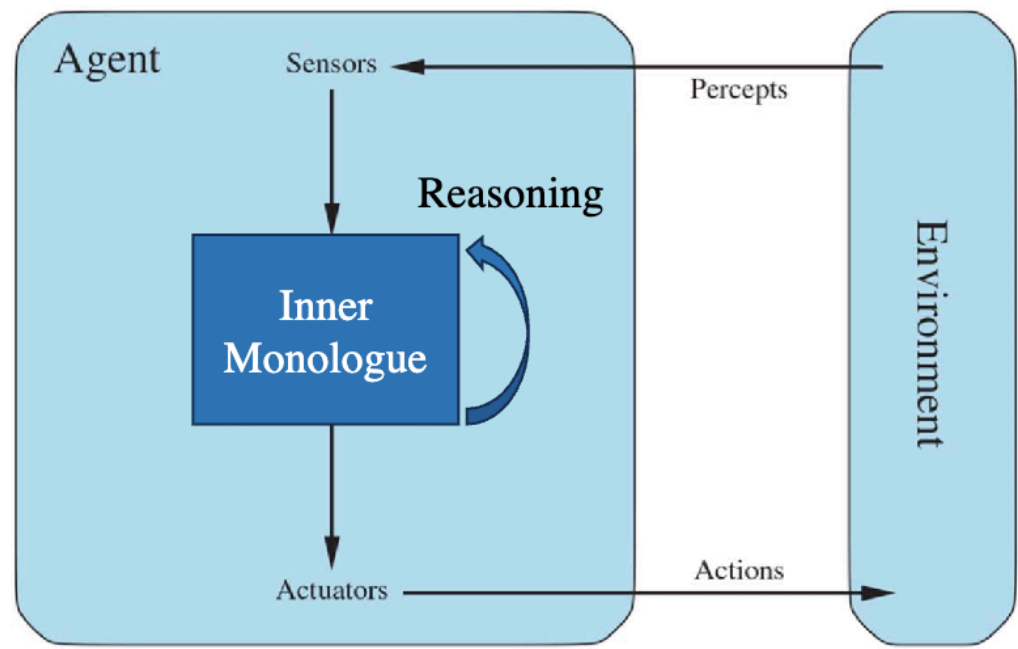
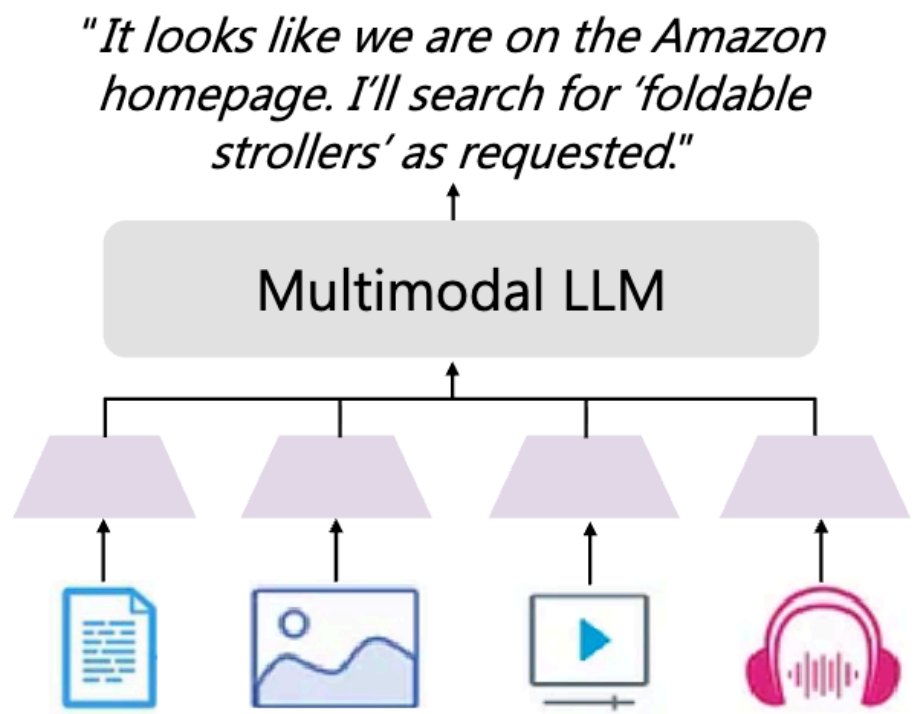
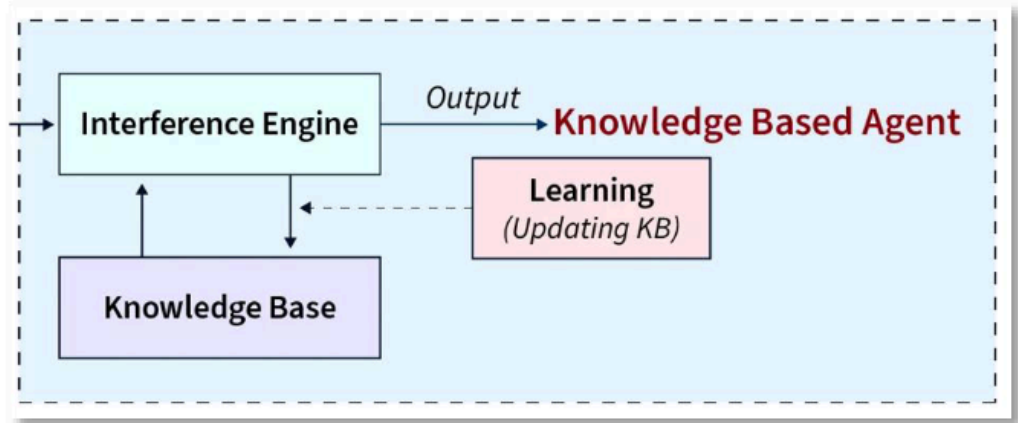
Language agent ဆိုတာ ဘာလဲ။ (Cont'd)



Logical Agent

Neural Agent

Language Agent



- Tokens များ ထုတ်ခြင်းဖြင့် စဉ်းစားတွေးခေါ်ခြင်း (Reasoning) သည် action အသစ် တစ်မျိုးဖြစ်
- ပြင်ပ environment များတွင် လုပ်ဆောင်သော actions များနှင့် ကွာခြားသည်
- မိမိကိုယ်ကိုယ် ရှေ့တွင် (inner monologue) ဖြင့် reasoning ပြုလုပ်သည့် Internal environment

Language Models

$$P(S) = \prod_{i=1}^n P(w_i | w_{i-1})$$

- ဘာသာစကား မော်ဒယ် language model (LM) ဟာ အရင်စကားလုံးတွေကို အခြေခံပြီး နောက်ထပ်စကားလုံးကို ခန့်မှန်းပါတယ်
- RNNs (Recurrent Neural Networks) တွေက အရှေ့က အချက်အလက်တွေကို hidden state ကို အသုံးပြုပြီး မှတ်သားထားနိုင်တယ်
- ရိုးရှင်းတဲ့ feed-forward network နဲ့မတူဘဲ၊ RNN က အချက်အလက်တွေကို စကားလုံးတစ်လုံးချင်းစီက ယူပါတယ်။
- သူ့ရဲ့ မှတ်ဉာဏ် (hidden state) ကို update လုပ်ပါတယ်။
- အဲဒီမှတ်ဉာဏ်ကို အသုံးပြုပြီး နောက်ထပ်စကားလုံးကို ခန့်မှန်းပါတယ်။
- အားနည်းချက် - Context ရှည်ရင် မမှတ်နိုင်



Bengio

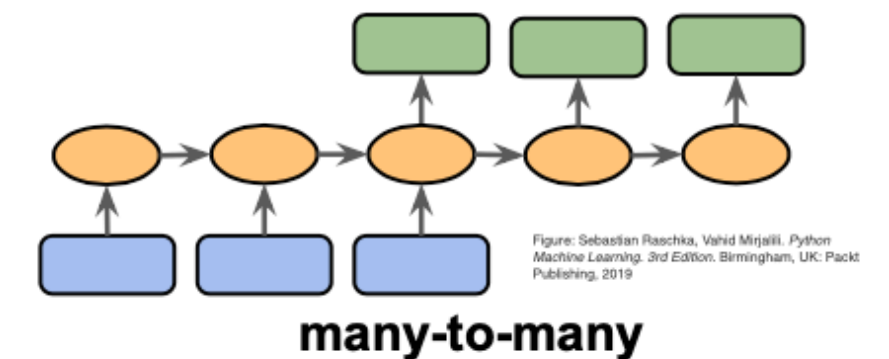
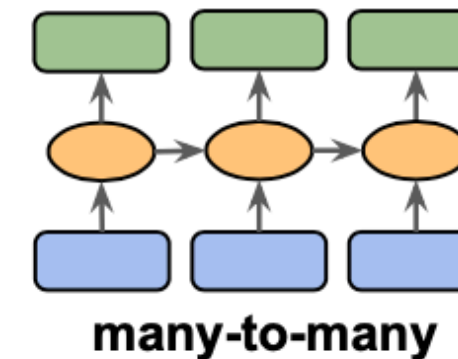
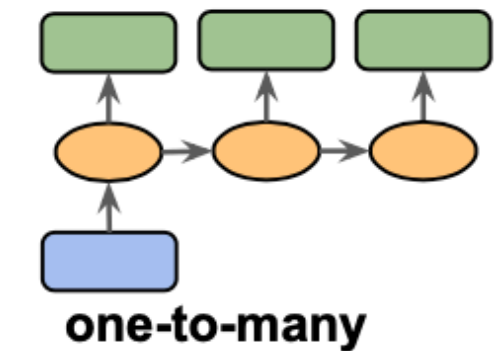
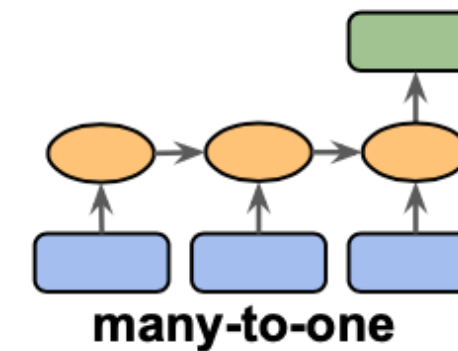
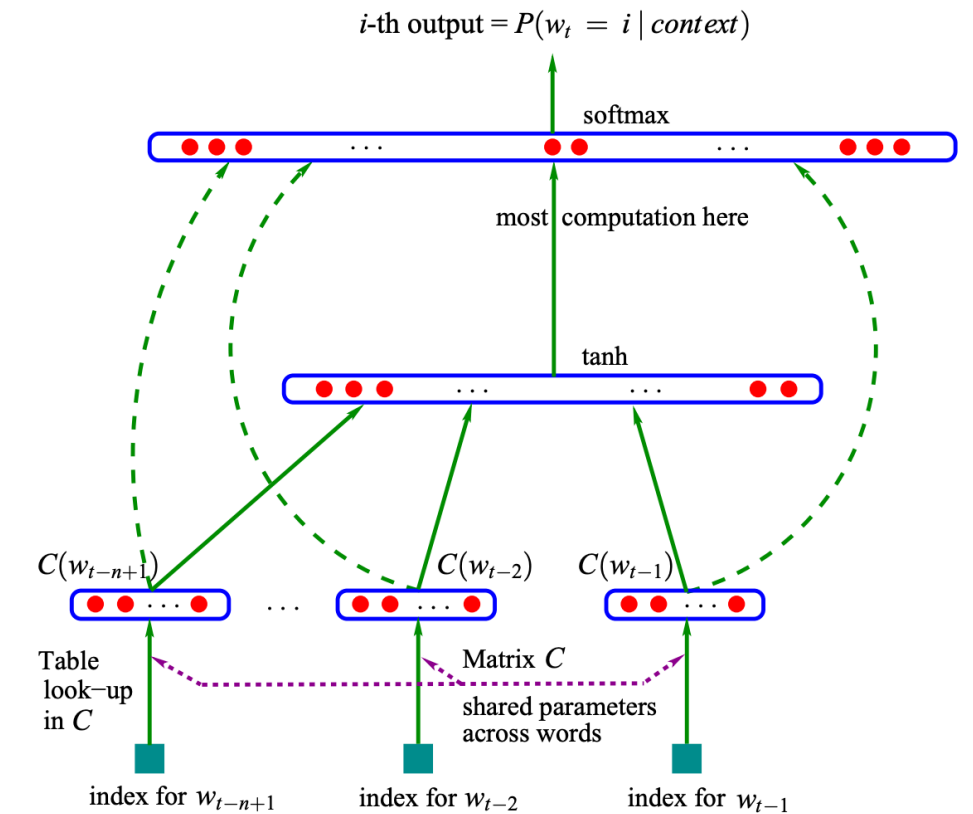


Figure: Sebastian Raschka, Vahid Mirjalili, Python Machine Learning, 3rd Edition, Birmingham, UK: Packt Publishing, 2019

Polysemy

စကားလုံး/token တစ်ခုမှာ အဓိပ္ပာယ်တစ်ခုထက်ပိုရှိနေခြင်း

Homonymy သံတူကြောင်းကွဲမဟုတ်

ဥပမာ

မင်း



- ကြံယာထောက်ပစ္စည်း၊ လွန်ကဲခြင်းအနက်ဖြင့်ကြံယာကိုထောက်ပံ့သောစကားလုံး → အိုမင်း၊ လွန်မင်း၊ ကြောက်မင်း။
- နာမ်စား၊ ရာထူးကြီးမြင့်သူကမိမိအောက်မှငယ်သားတို့အားရည်ညွှန်းသောစကားလုံး → မင်းသိသမျှ ငါ့ကို ပြော။
- နာမ်စား၊ မိမိကဦးတည်ပြောဆိုသူကိုရည်ညွှန်းသောစကားလုံး → သူငယ်ချင်း၊ မင်းနဲ့ ငါ အတူသွားမယ်နော်။
- နာမဝိသေသန၊ ဂုဏ်ပြု ချီးမြှင့်ခေါ်သော စကားလုံး → လူကြီးမင်း၊ ဦးမင်း။
- နာမ်၊ အစိုးရအမှုထမ်းအကြီးအကဲ → တပ်မင်း၊ ဗိုလ်မင်း အပေါင်းတို့ သည်လည်း။
- နာမ်၊ ဘုရင့်ဆွေတော်မျိုးတော် → သားတော်အိမ်ရှေ့မင်း။
- နာမ်၊ ဘုရင် → မင်းတို့စကား၊ မိုးကြိုးသွား။

Transformer Embeddings

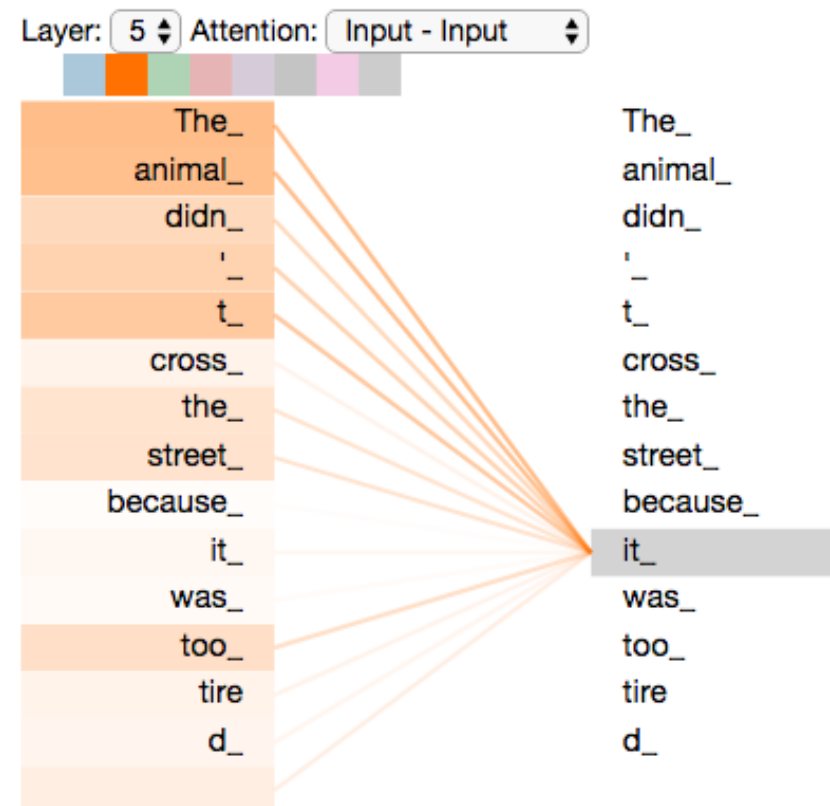
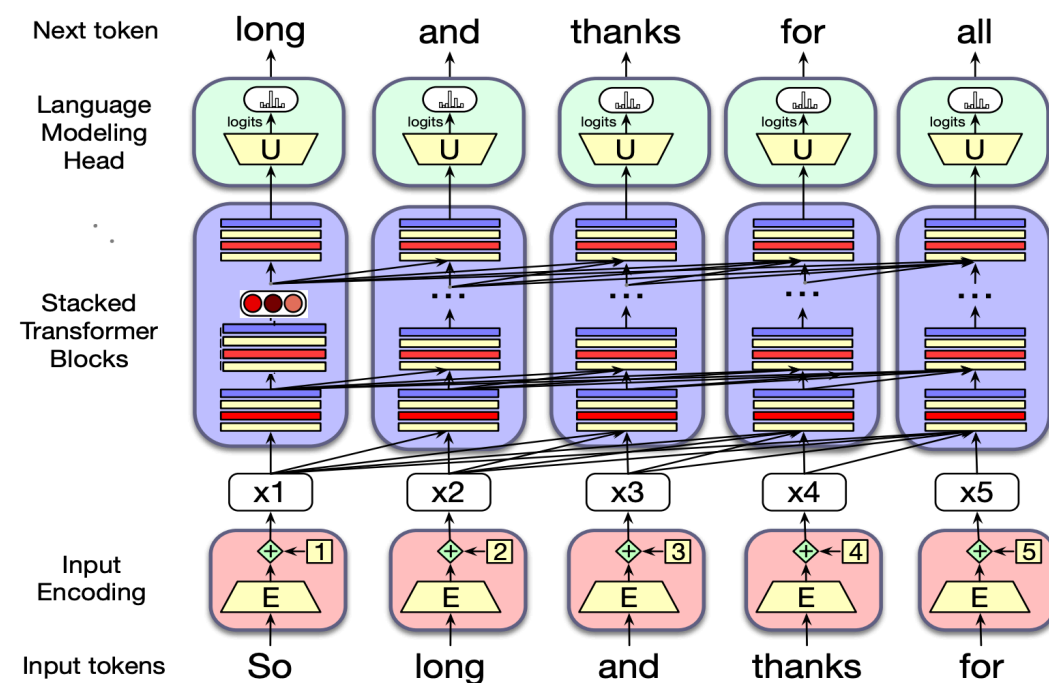
$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) V$$



Noam Shazeer



Ashish Vaswani



- Self-attention - တစ်ခုနဲ့တစ်ခု ပုံပြီး ဆင်တူတဲ့ tokens တွေကို အဓိပ္ပာယ်တစ်ခုတည်းဆီကို ရောက်အောင်ကြိုးစားခြင်း
- အစောပိုင်း context tokens တွေက နောက်ပိုင်း tokens တွေရဲ့ embedding representations ကို သက်ရောက်မှုရှိပါတယ်
- သိအိုရီအရ context token တစ်ခုစီဟာ စာကြောင်းတစ်ခုလုံးရဲ့ အဓိပ္ပာယ်ကို ထည့်သွင်းနိုင်ပါတယ်
- Transformers တွေက tokens တွေကို တပြိုင်နက်တည်း လုပ်ဆောင်ပါတယ် (တစ်လုံးပြီးမှတစ်လုံးဖတ်တဲ့ RNNs တွေနဲ့မတူပါဘူး)
- Problem → Transformers တွေက စကားလုံးအစီအစဉ်ကို မူလကတည်းက မသိပါ
- Solution → Positional Encoding: မော်ဒယ်က tokens တွေရဲ့ အစီအစဉ်ကို သိအောင်လို့ input embeddings တွေထဲကို positional signal တစ်ခု ထည့်ပေးရပါမယ်

<https://jalammar.github.io/illustrated-transformer/>

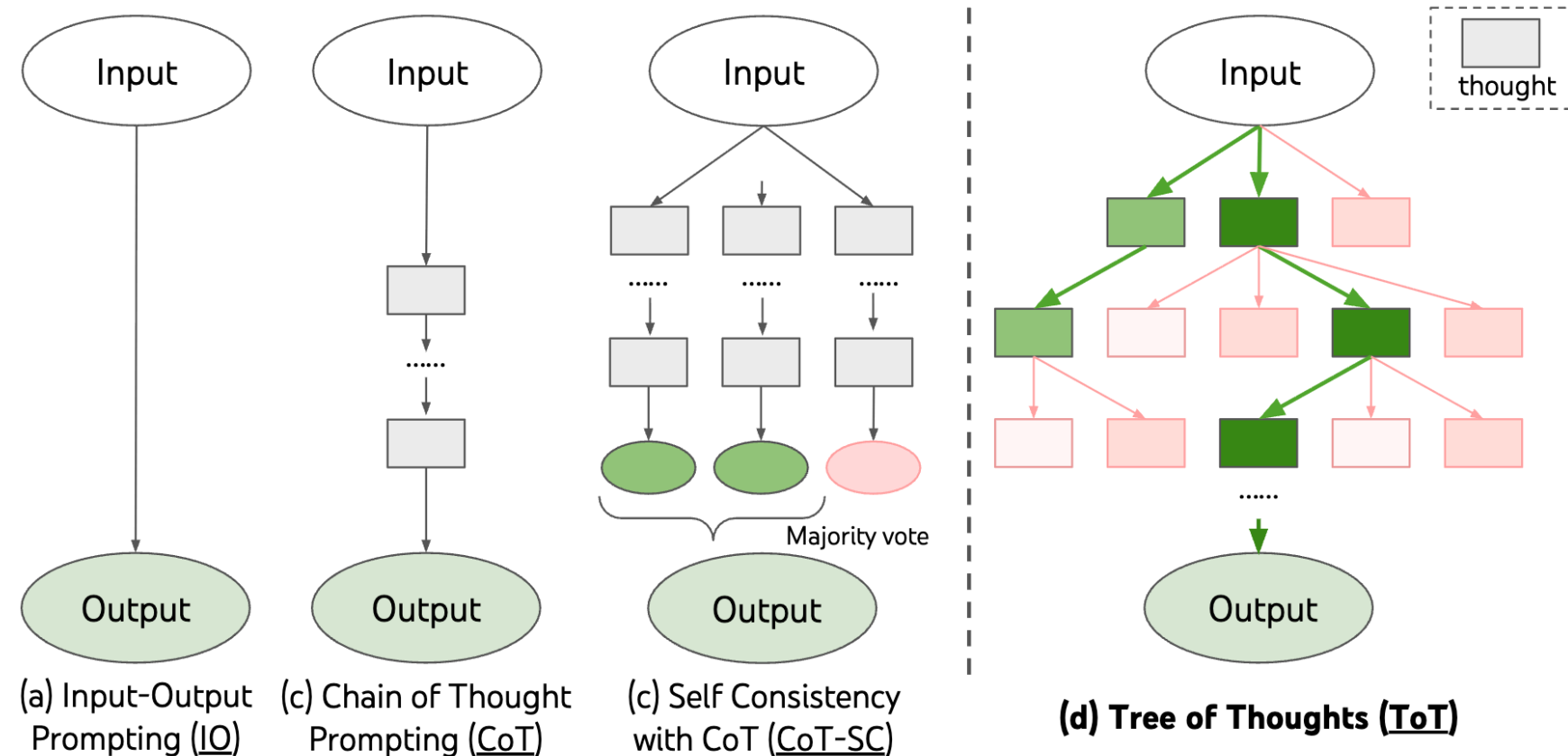
<https://web.stanford.edu/~jurafsky/slp3/slides/transformer24aug.pdf>

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017).

Attention is all you need. In Advances in Neural Information Processing Systems (pp. 5998–6008).

Context and Thinking

- **Zero-shot Learning:** Model ကို ဘာမှမဟုတ် (Example) မှ ကြိုမပြဘဲ တန်းခိုင်းတာ။
- **In-context Learning (ICL):** Prompt (စာသား) ထဲမှာ နမူနာတွေ ထည့်ပေးပြီး Model ကို အလိုက်သင့် လိုက်လုပ်ခိုင်းတာ။ (OpenAI)
- **RAG:** ပိုပြီး တိကျသေချာအောင် ပြင်ပ Data အချက်အလက်တွေကို ရှာဖွေပြီးမှ အဖြေ ထုတ်ပေးတာ။ (Meta)



Tom Brown



Benjamin Mann



Nick Ryder

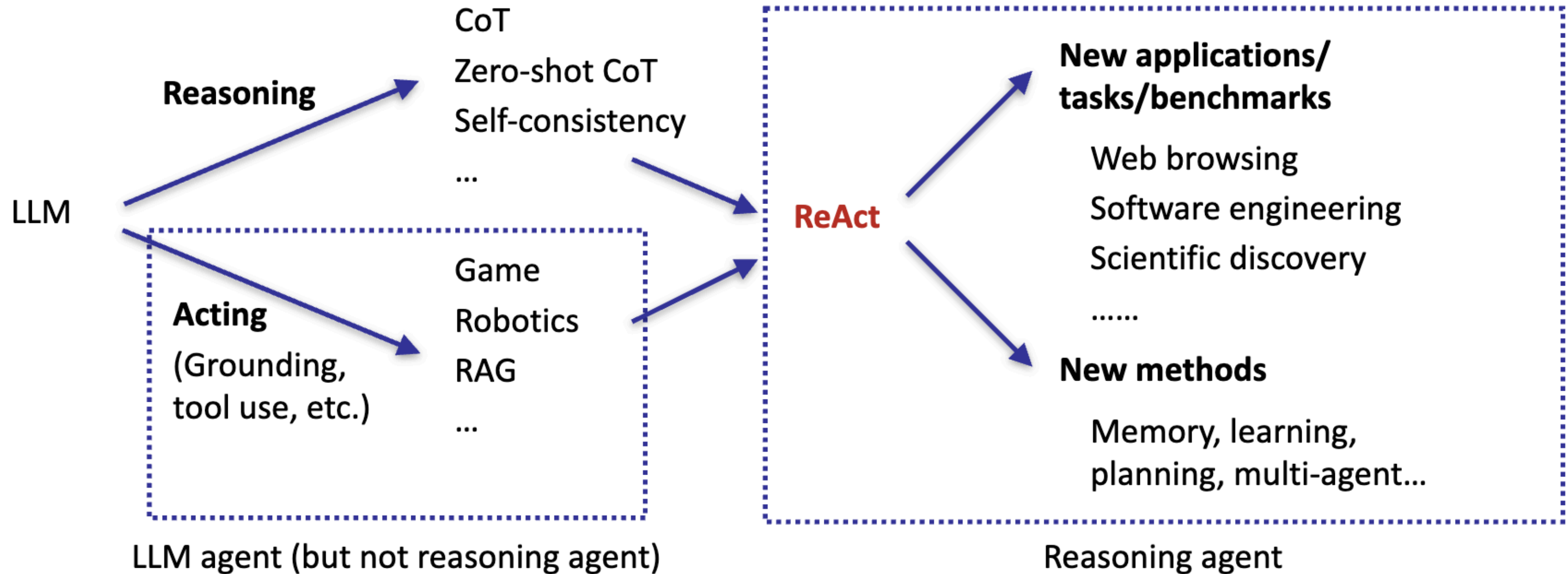


Melanie Subbiah



Patrick Lewis and Douwe
Kiela

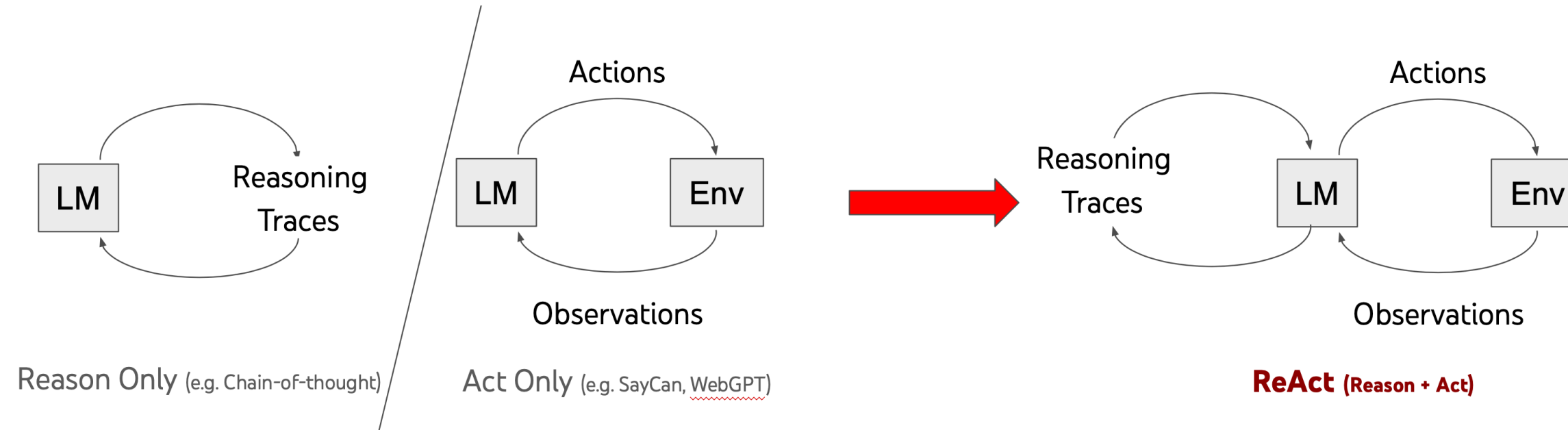
Reasoning and Action



ReAct



Shunyu Yao



၁။ Reason (စဉ်းစားခြင်း) AI က သူ့ကိုပေးထားတဲ့ မေးခွန်း ဒါမှမဟုတ် အလုပ်တစ်ခုကို အရင်ဆုံး သုံးသပ်ပါတယ်။ ပြဿနာရဲ့ အရင်းအမြစ်ကဘာလဲ၊ ဘာတွေလုပ်ဖို့လိုလဲဆိုတာကို သူ့ရဲ့ ဗဟုသုတကို အခြေခံပြီး စဉ်းစားပါတယ်။

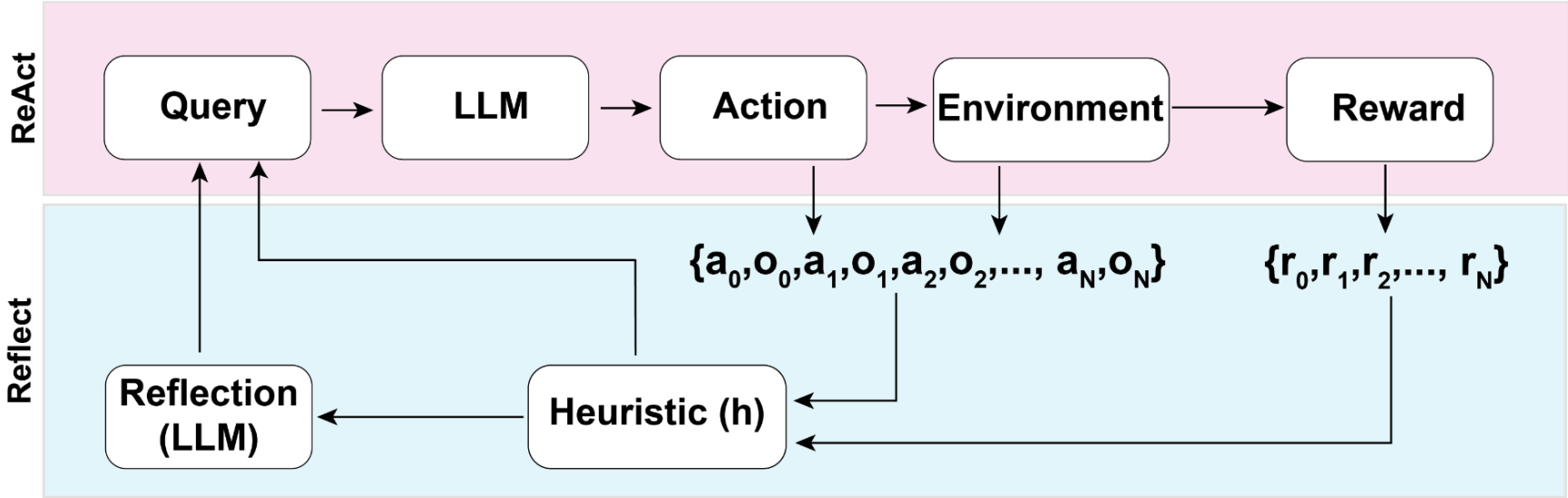
၂။ Act (လုပ်ဆောင်ခြင်း) စဉ်းစားပြီးတဲ့နောက်မှာ အဖြေရှာဖို့အတွက် အပြင်လောကမှာ တကယ်လှုပ်ရှားလုပ်ဆောင်ပါတယ်။ ဥပမာ - အင်တာနက်ကနေ သတင်းအချက်အလက်အသစ်တွေ ရှာဖွေတာ (သို့) တွက်ချက်မှုတွေ ပြုလုပ်တာမျိုး ဖြစ်ပါတယ်။

စဉ်းစားတယ် Thought → အဖြေထုတ်ဖို့ တစ်ခုခုကို လုပ်ဆောင်တယ် Action → လုပ်ဆောင်ချက်ကနေ ရလာတဲ့ အဖြေ၊ အချက်အလက်ကို ကြည့်တယ် Observation → ရလာတဲ့ အချက်အလက်အသစ်အပေါ်မူတည်ပြီး နောက်ထပ် ဘာလုပ်ရမလဲဆိုတာကို ထပ်စဉ်းစားတယ်။

Self-Reflection (Reflexion)

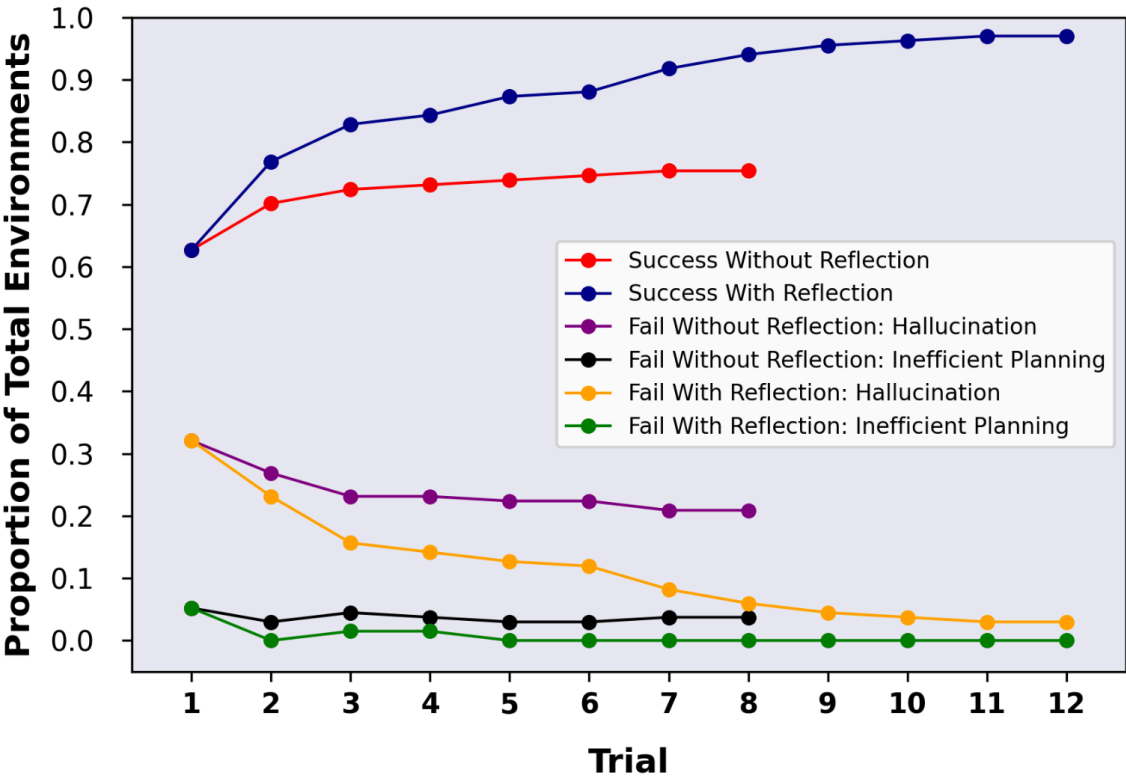


Noah Shinn



Heuristic AI ရဲ့ အဖြေတွေကို ကြိုသတ်မှတ်ထားတဲ့ စနစ်နဲ့ Reward Score ပေးပြီး၊ အမှတ်မကောင်းရင် Critique ထုတ်ပေးပြီး အမှားပြင်ခိုင်းတာပါ။

- ၁။ အမှားကို မှတ်ထားတယ် (Dynamic Memory)
AI က သူ့အရင်က ဘာတွေလုပ်ခဲ့လဲ၊ ဘယ်နေရာမှာ မှားခဲ့လဲဆိုတာကို ခေါင်းထဲမှာ မှတ်ထားနိုင်တဲ့ မှတ်ဉာဏ် ရှိသွားတယ်။
- ၂။ ကိုယ်တိုင် ပြန်ဆန်းစစ်တယ် (Self-Critique)
အလုပ်တစ်ခုခု မှားသွားရင် သူများ (လူ) လာပြောမှ သိတာမဟုတ်ဘဲ "ငါ ဘာတွေ လွဲသွားပါလိမ့်" ဆိုပြီး AI ကိုယ်တိုင် ပြန်ဝေဖန် ဆန်းစစ်တယ်။

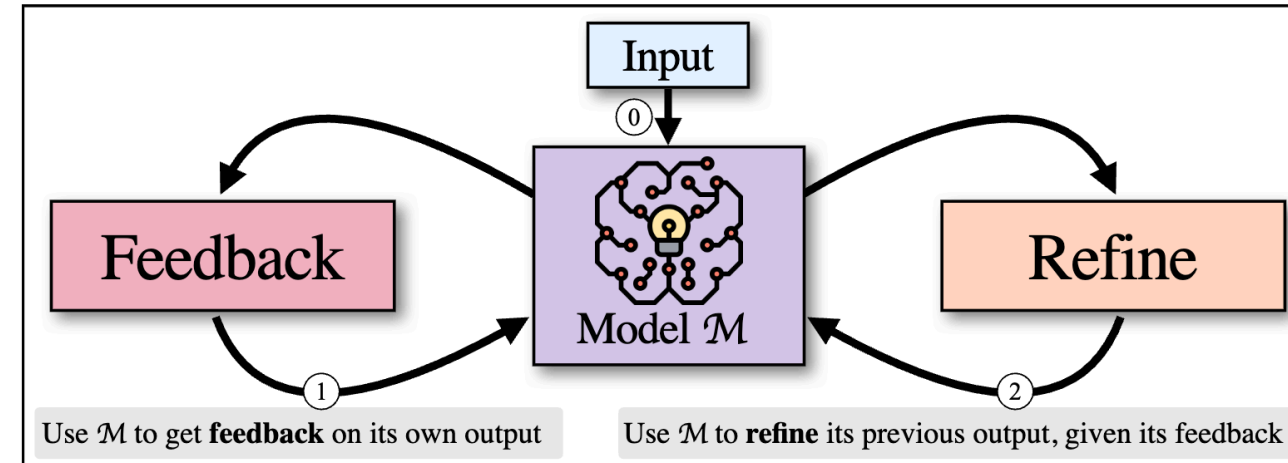


Self-Refine



Aman Madaan

- Self-Refine (အမှားရှာပြီး ချက်ချင်းပြင်တာ)
- Self-Reflection (အတိတ်က အမှားကို သင်ခန်းစာယူတာ)



Task	GPT-3.5		ChatGPT		GPT-4	
	Base	+SELF-REFINE	Base	+SELF-REFINE	Base	+SELF-REFINE
Sentiment Reversal	8.8	30.4 (↑21.6)	11.4	43.2 (↑31.8)	3.8	36.2 (↑32.4)
Dialogue Response	36.4	63.6 (↑27.2)	40.1	59.9 (↑19.8)	25.4	74.6 (↑49.2)
Code Optimization	14.8	23.0 (↑8.2)	23.9	27.5 (↑3.6)	27.3	36.0 (↑8.7)
Code Readability	37.4	51.3 (↑13.9)	27.7	63.1 (↑35.4)	27.4	56.2 (↑28.8)
Math Reasoning	64.1	64.1 (0)	74.8	75.0 (↑0.2)	92.9	93.1 (↑0.2)
Acronym Generation	41.6	56.4 (↑14.8)	27.2	37.2 (↑10.0)	30.4	56.0 (↑25.6)
Constrained Generation	28.0	37.0 (↑9.0)	44.0	67.0 (↑23.0)	15.0	45.0 (↑30.0)

Table 1: SELF-REFINE results on various tasks using GPT-3.5, ChatGPT, and GPT-4 as base LLM. SELF-REFINE consistently improves LLM. Metrics used for these tasks are defined in Section 3.2.

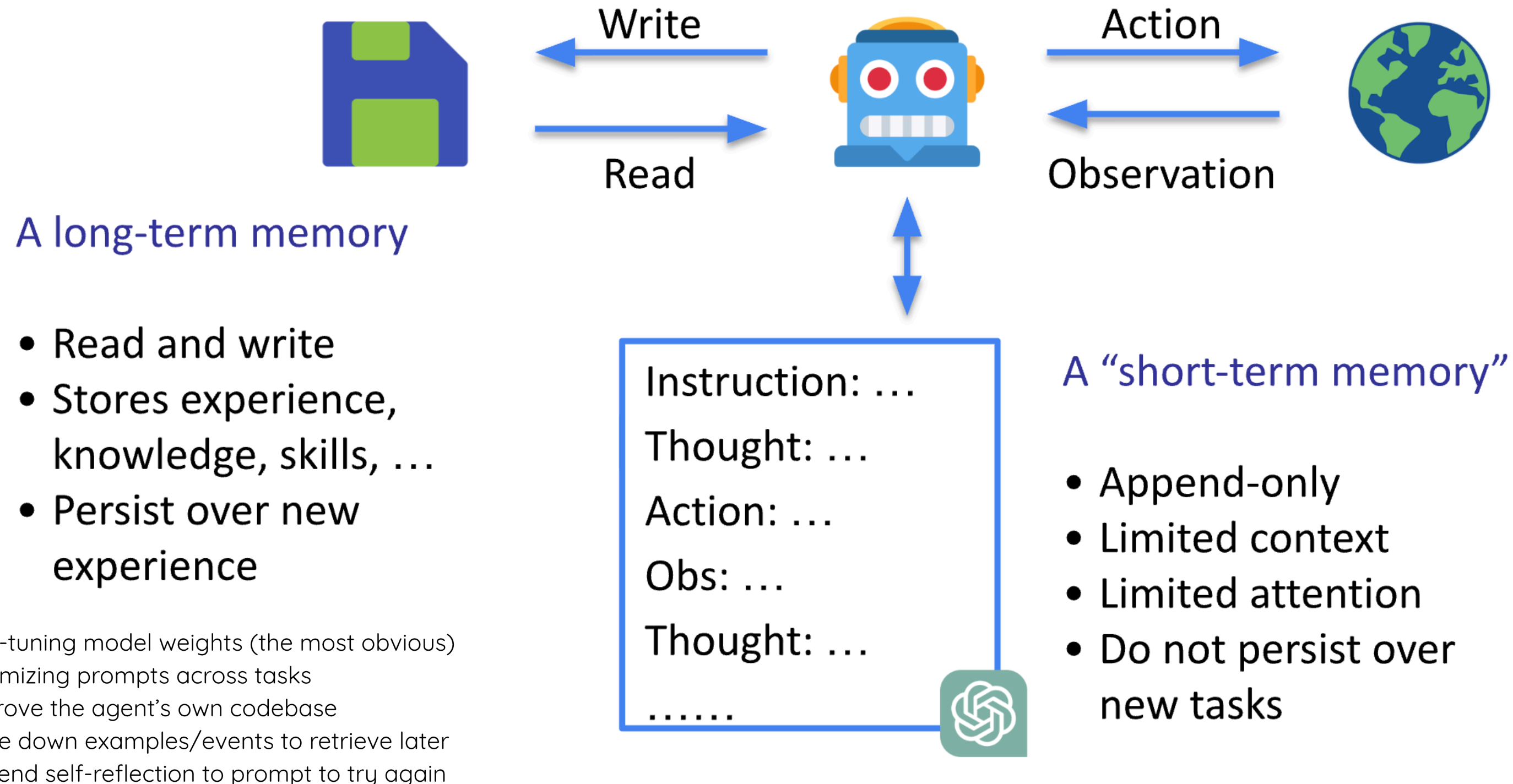
၁။ Outputs (အဖြေထုတ်ခြင်း) AI က သူ့ကိုပေးထားတဲ့ Task အတွက် အဖြေတစ်ခုကို အရင်ဆုံး ထုတ်ပေးပါတယ်။

၂။ Critique (ကိုယ်တိုင် ဝေဖန်ဆန်းစစ်ခြင်း) ထွက်လာတဲ့ အဖြေကို AI က ပြန်ဖတ်ပြီး သူ့ဘာသာသူ Feedback ပြန်ပေး၊ ပြန်ဝေဖန်ပါတယ်။

၃။ Refine (အမှားပြင်ဆင် မွမ်းမံခြင်း) သူ့ဘာသာသူ ပေးထားတဲ့ ဝေဖန်ချက် (Critique) အပေါ်မူတည်ပြီး အဖြေကို ပိုကောင်းအောင် ထပ်ဆင့် ပြုပြင်မွမ်းမံပါတယ်။

Memory

Type by content	Definition	Examples
Episodic memory	Stores experience	Generative agents [Park et al., 2023]
Semantic memory	Stores knowledge	
Procedural memory	Stores skills	Voyager [Wang et al., 2023]



Memory (Cont'd)

Episodic memory

- Write: append-only event streams
- Read: retrieval based on heuristic scores

Memory Stream

2023-02-13 22:48:20: desk is idle

2023-02-13 22:48:20: bed is idle

2023-02-13 22:48:10: closet is idle

2023-02-13 22:48:10: refrigerator is idle

2023-02-13 22:48:10: Isabella Rodriguez is stretching

2023-02-13 22:33:30: shelf is idle

2023-02-13 22:33:30: desk is neat and organized

2023-02-13 22:33:10: Isabella Rodriguez is writing in her journal

2023-02-13 22:18:10: desk is idle

2023-02-13 22:18:10: Isabella Rodriguez is taking a break

2023-02-13 21:49:00: bed is idle

2023-02-13 21:48:50: Isabella Rodriguez is cleaning up the kitchen

2023-02-13 21:48:50: refrigerator is idle

2023-02-13 21:48:50: bed is being used

2023-02-13 21:48:10: shelf is idle

2023-02-13 21:48:10: Isabella Rodriguez is watching a movie

2023-02-13 21:19:10: shelf is organized and tidy

2023-02-13 21:18:10: desk is idle

2023-02-13 21:18:10: Isabella Rodriguez is reading a book

2023-02-13 21:03:40: bed is idle

2023-02-13 21:03:30: refrigerator is idle

2023-02-13 21:03:30: desk is in use with a laptop and some papers on it

...

Q. What are you looking forward to the most right now?

Isabella Rodriguez is excited to be planning a Valentine's Day party at Hobbs Cafe on February 14th from 5pm and is eager to invite everyone to attend the party.

retrieval

2.34

=

recency

0.91

+

importance

0.63

+

relevance

0.80

ordering decorations for the party

2.21

=

0.87

+

0.63

+

0.71

researching ideas for the party

2.20

=

0.85

+

0.73

+

0.62

...

I'm looking forward to the Valentine's Day party that I'm planning at Hobbs Cafe!

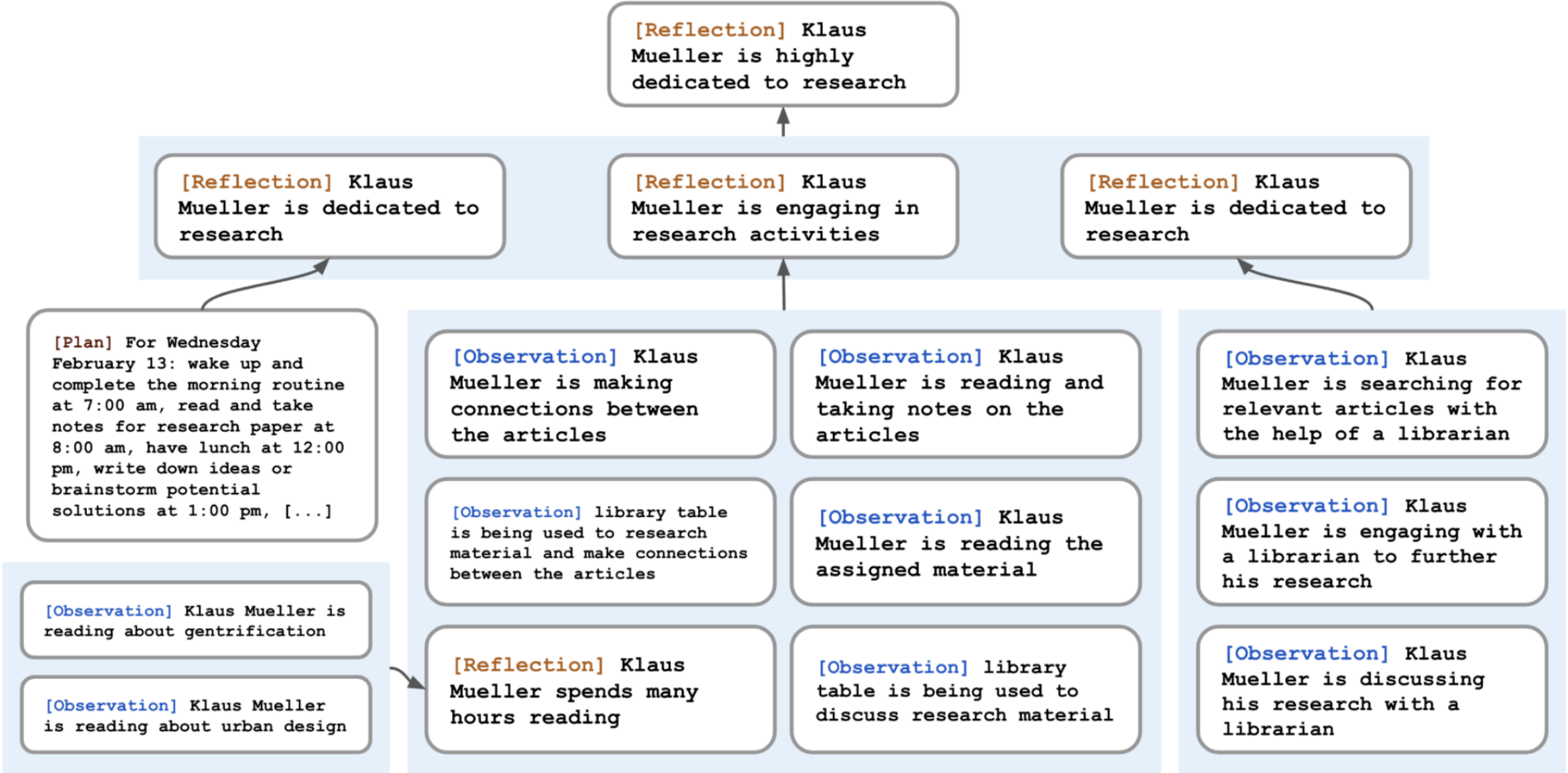


Isabella

Memory (Cont'd)

Semantic memory

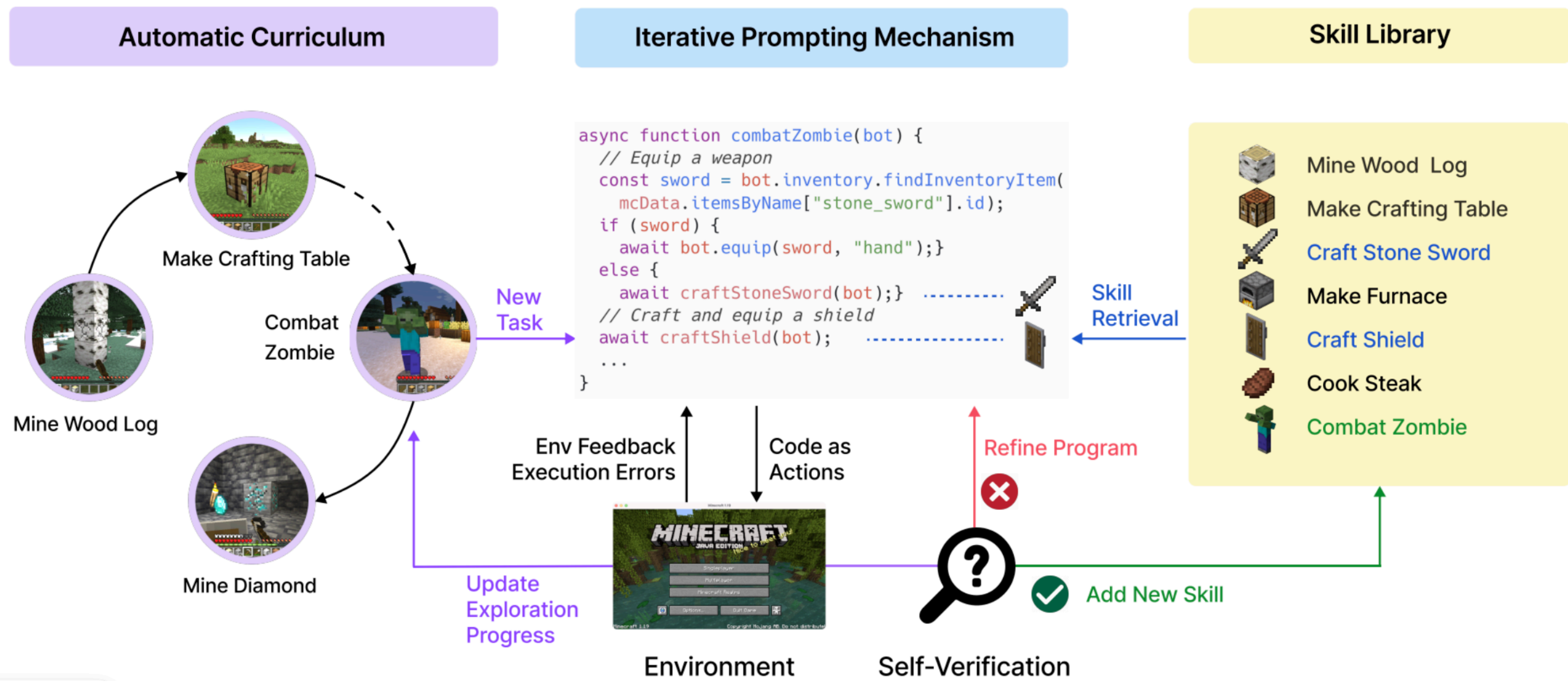
- Write: LLM reasoning over events
- Read: retrieval



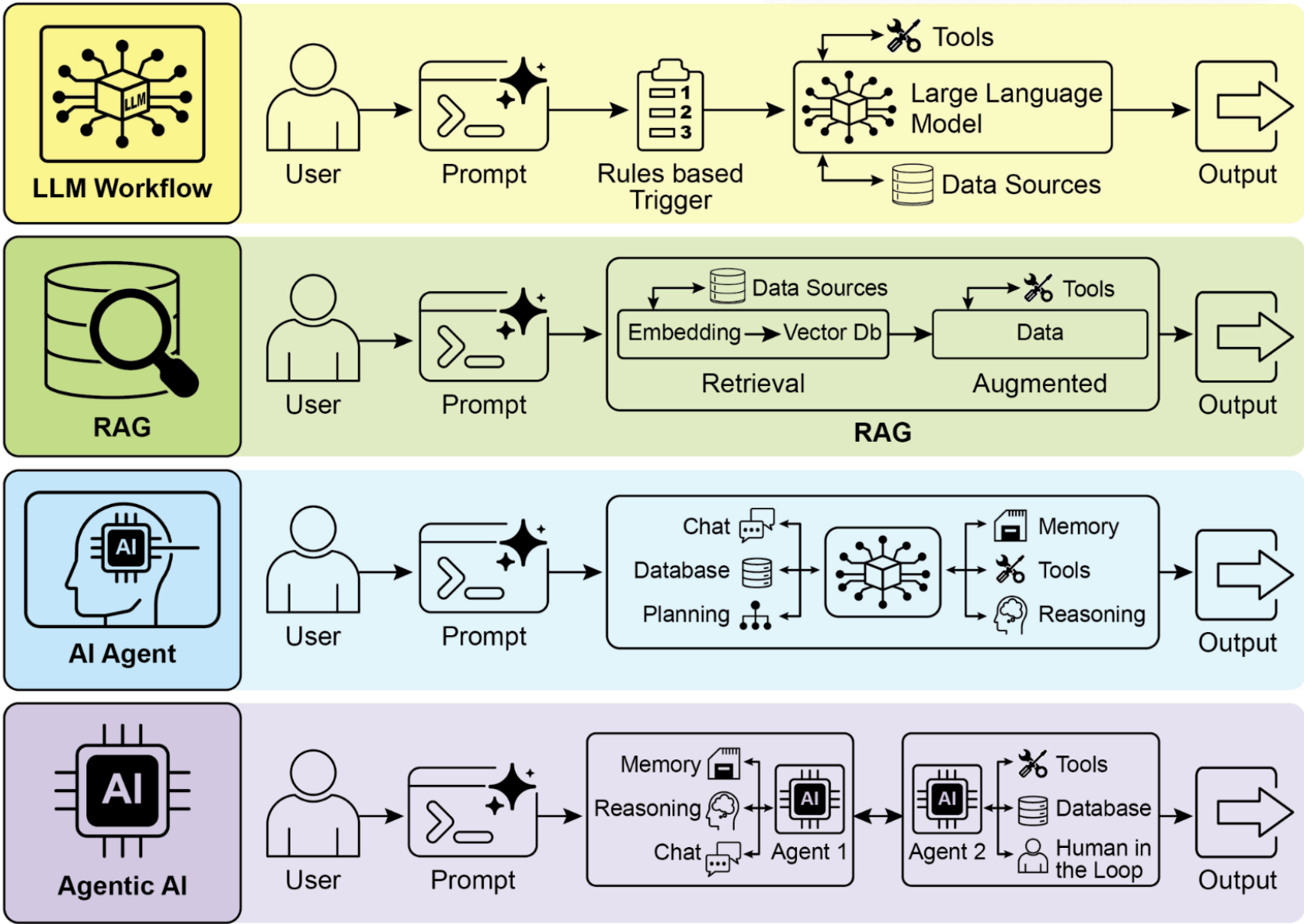
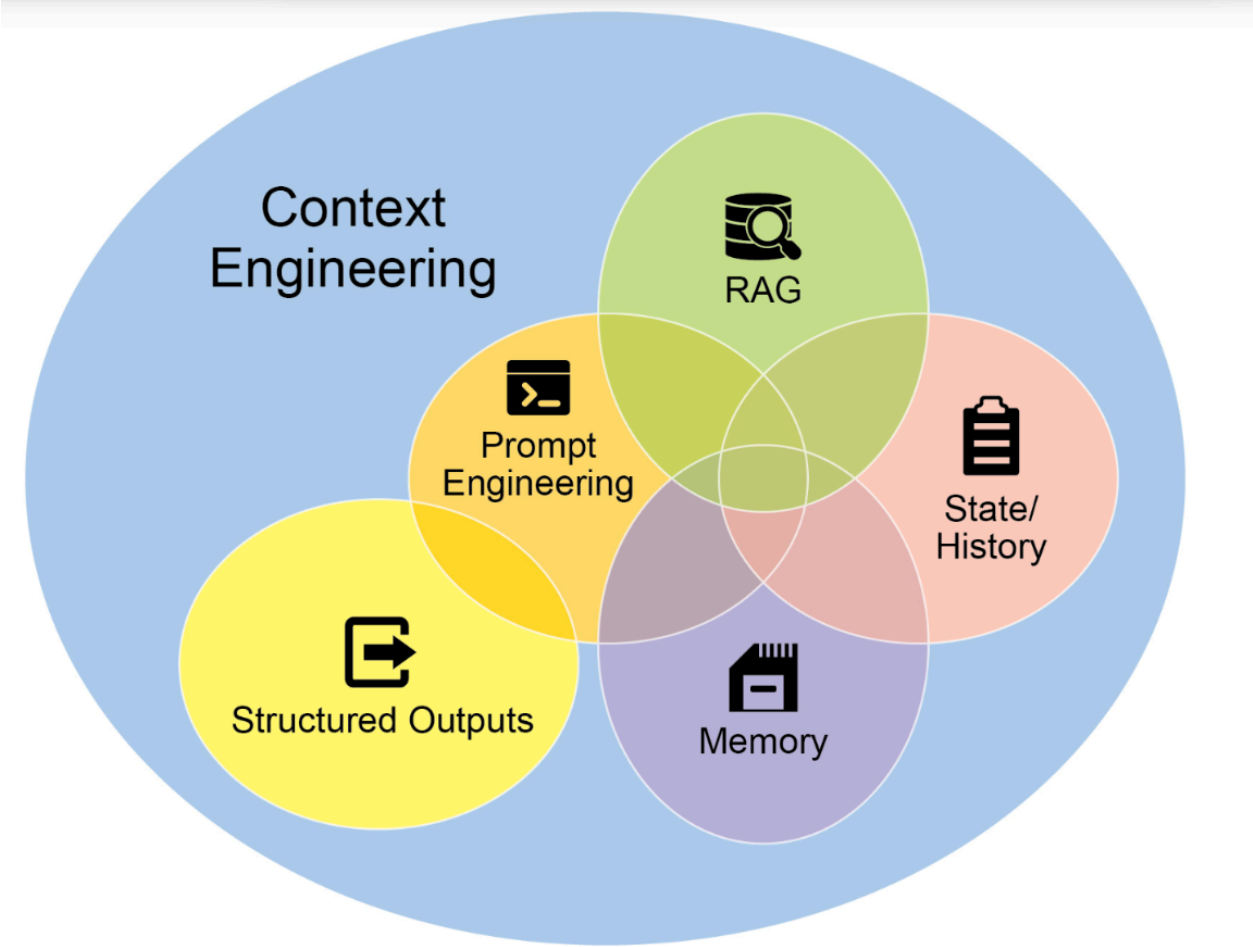
Memory (Cont'd)

Voyager: Procedural memory

- Write: Code-based skills
- Read: Embedding retrieval

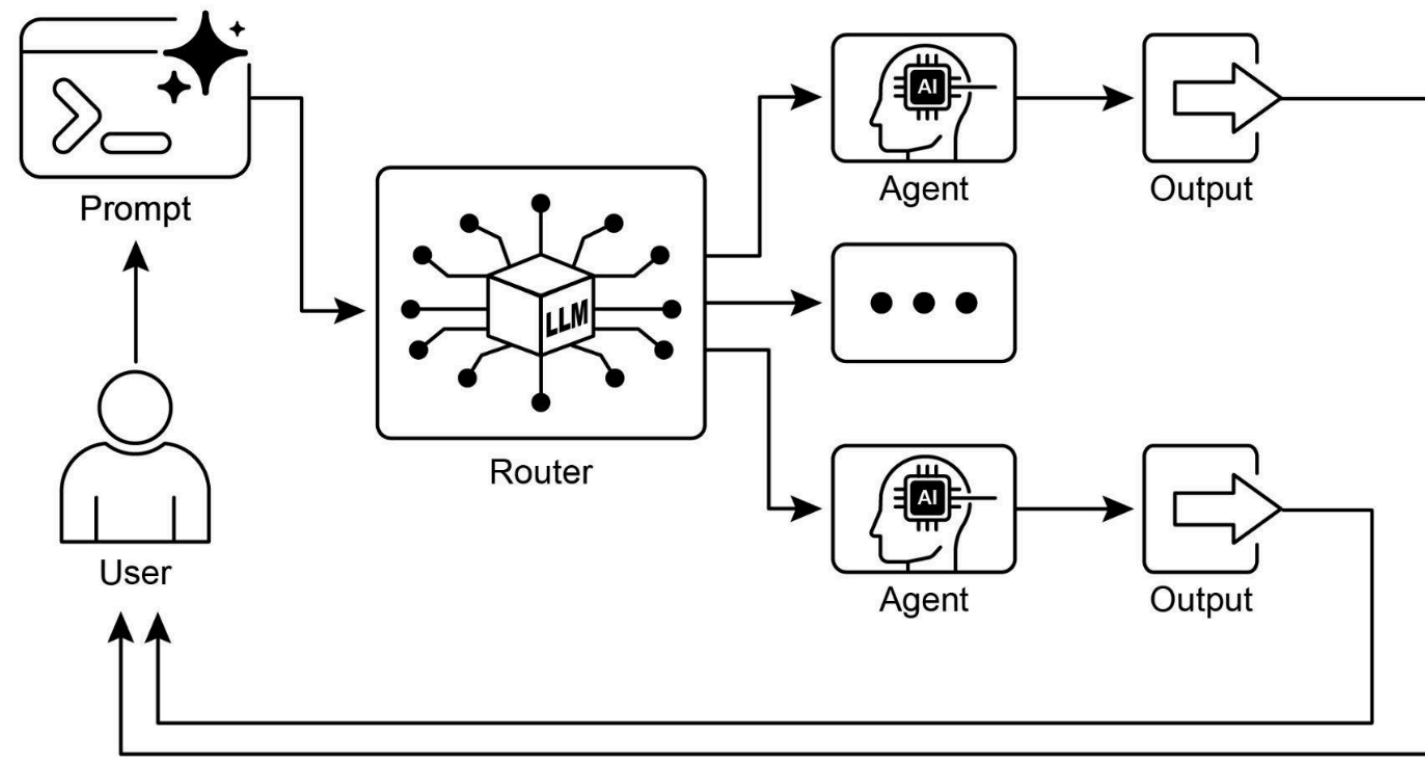


Agentic AI



Agentic AI: Router Pattern

AI တစ်ခုတည်းကို အလုပ်တွေအကုန် ဇွတ်ခိုင်းမယ့်အစား ဒီ Router က အလုပ်တွေကို သင့်တော်တဲ့ အထူးပြု Agent တွေဆီ ခွဲဝေပေးတာ ဖြစ်ပါတယ်။

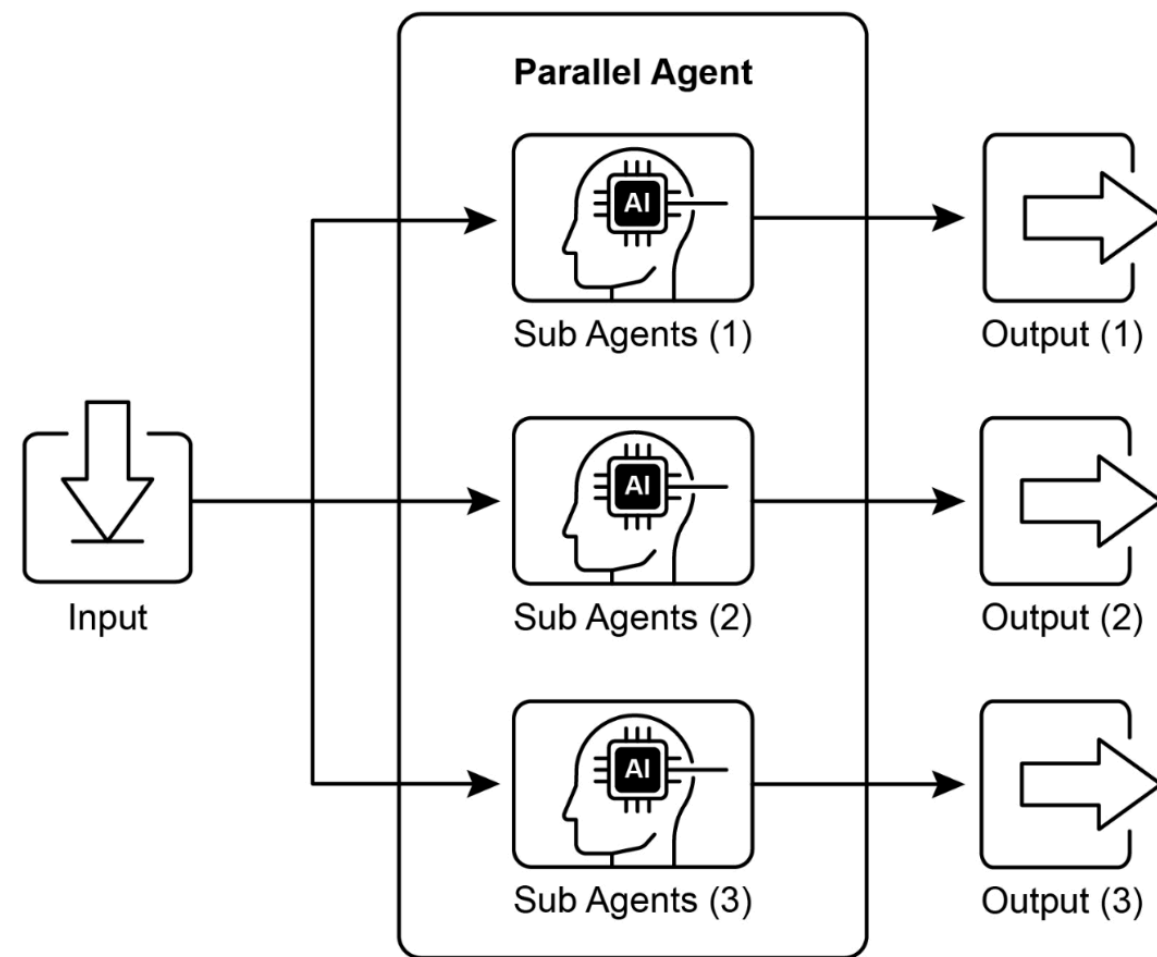


- အချက်အလက် စစ်ဆေးခြင်း: အသုံးပြုသူ (User) က မေးခွန်းမေးလိုက်တာနဲ့ Router က "သူ ဘာကို ဆိုလိုတာလဲ (Intent)" ဆိုပြီး အရင်ဆုံး ဖတ်ကြည့်ပါတယ်။
- လမ်းခွဲပေးခြင်း: မေးခွန်း အမျိုးအစားအလိုက် သက်ဆိုင်ရာ Specialized Agent (အထူးပြု အေးဂျင့်) သို့မဟုတ် ကိရိယာ (Tool) ဆီကို လှမ်းပို့ပေးလိုက်ပါတယ်။
- အဖြေထုတ်ခြင်း: ထိုအထူးပြု အေးဂျင့်ကပဲ အလုပ်ကို အကောင်းဆုံး ဖြေရှင်းပြီး အဖြေပြန်ထုတ်ပေးပါတယ်။

Agentic AI: Router Pattern (Cont'd)

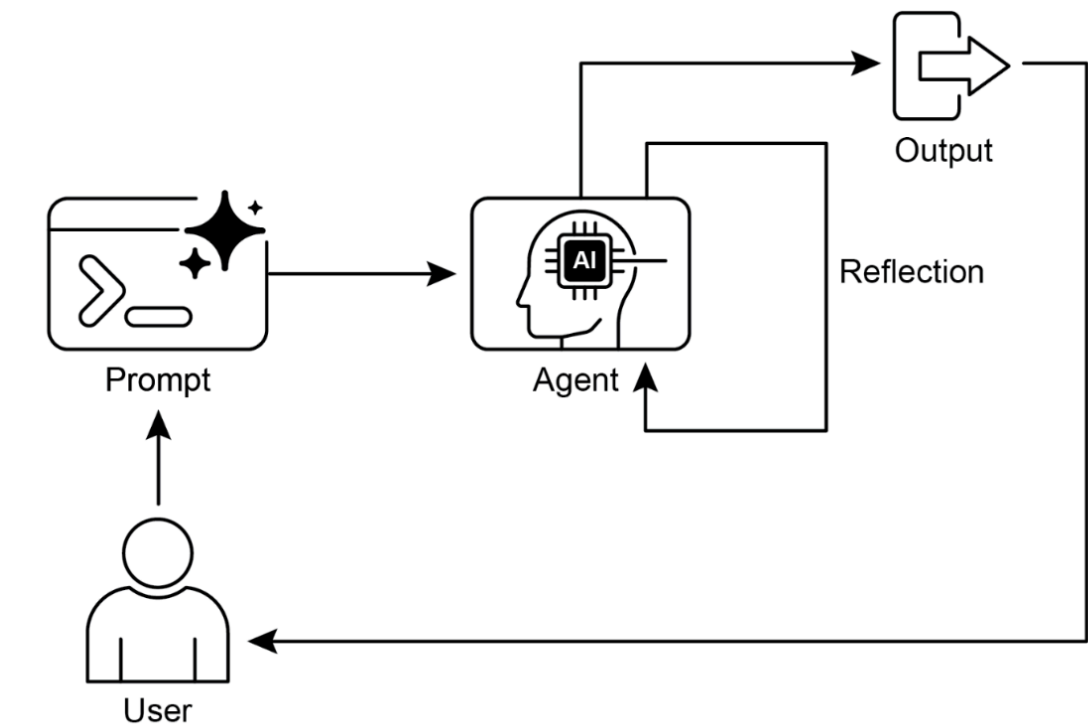
- **Model-based:** Router နေရာမှာ AI Model သေး (ဥပမာ- GPT-4o-mini, Claude Haiku) တစ်ခုကို ထားပြီး User ရဲ့ Prompt ကို ဖတ်ခိုင်းကာ ဘယ် Agent ဆီ သွားရမလဲဆိုတာကို JSON သို့မဟုတ် Text နဲ့ တိုက်ရိုက် ရွေးချယ်ခိုင်းတာ ဖြစ်ပါတယ်။
- **Embedding-based:** မေးခွန်းထဲက စကားလုံးတွေကို ကွန်ပျူတာနားလည်တဲ့ ကိန်းဂဏန်းတွေ (Vectors/Embeddings) အဖြစ် ပြောင်းလဲပြီး ကြိုတင်သတ်မှတ်ထားတဲ့ ခေါင်းစဉ်တွေနဲ့ သင်္ချာနည်းအရ မည်မျှနီးစပ်လဲ (Cosine Similarity) တွက်ချက်ကာ လမ်းခွဲတာ ဖြစ်ပါတယ်။ (ဥပမာ- Semantic Router ကဲ့သို့ Library များ သုံးခြင်း)
- **Rule-based:** ကြိုတင်သတ်မှတ်ထားတဲ့ Keyword (သို့) Regex (Regular Expressions) တွေပေါ်မူတည်ပြီး Hardcode ရေးပြီး ခွဲတာ ဖြစ်ပါတယ်။
- **Auction-based:** ဗဟိုကနေ လမ်းခွဲမပေးဘဲ အောက်ခြေက Specialized Agent အားလုံးဆီ မေးခွန်းကို အရင်ပြလိုက်ပါတယ်။ ပြီးရင် Agent တစ်ခုစီကို "မင်း ဒီအလုပ်ကို ဘယ်လောက်အထိ ကောင်းကောင်းလုပ်နိုင်လဲ" ဆိုပြီး Confidence Score (ယုံကြည်မှု ရမှတ်) တောင်းပါတယ်။ အမှတ်အများဆုံး ပေးနိုင်တဲ့ Agent (Highest Bidder) ကို အလုပ်လွှဲပေးတဲ့ စနစ် ဖြစ်ပါတယ်။

Agentic AI: Parallel Pattern

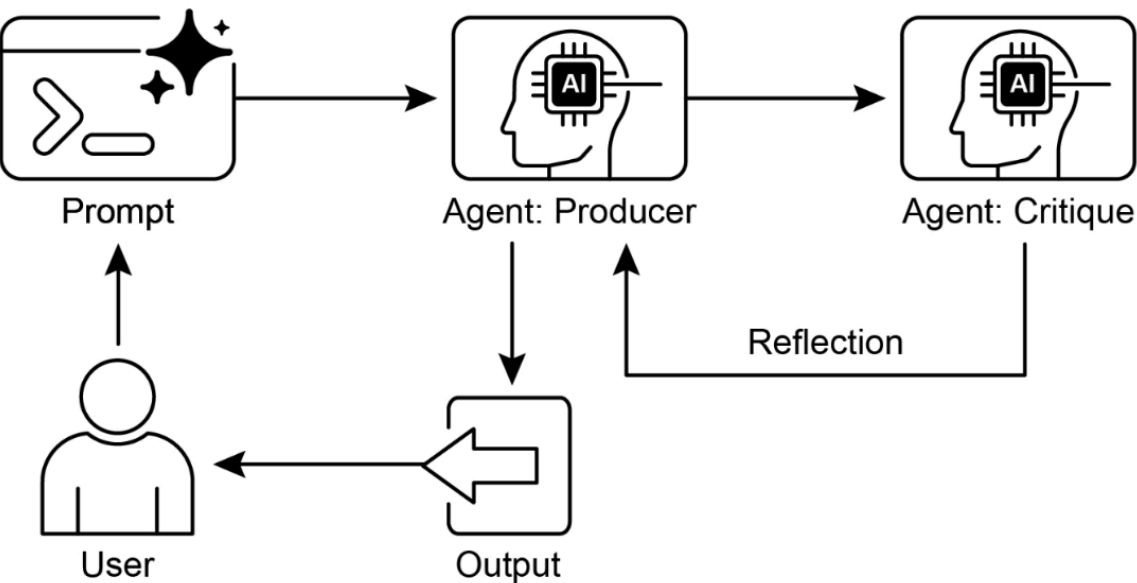


- **Sectioning/ Sub-tasking:** ကြီးမား ရှုပ်ထွေးတဲ့ အလုပ်ကြီးတစ်ခုကို သီးခြားလွတ်လပ်တဲ့ အပိုင်းအစလေးတွေ (Sub-tasks) ခွဲပစ်လိုက်ပါတယ်။ ပြီးရင် အဲဒီအပိုင်းတွေကို Agent တစ်ခုစီဆီ တစ်ပြိုင်တည်း ပို့ပေးပြီး အလုပ်လုပ်ခိုင်းတာ ဖြစ်ပါတယ်။
- **Voting/ Ensemble:** ဒီစနစ်မှာတော့ အလုပ်ကို အပိုင်းမခွဲပါဘူး။ မေးခွန်း (သို့) အလုပ်တစ်ခုတည်းကိုပဲ မတူညီတဲ့ AI Agents ၃/၄ ခုဆီကို တစ်ပြိုင်တည်း ပို့လိုက်ပါတယ်။ ပြီးရင် အဲဒီ Agent တွေဆီက ရလာတဲ့ အဖြေတွေကို ဗဟိုစနစ် (Synthesizer) က ပြန်စုစည်းပြီး "ဘယ်အဖြေက အများဆုံး တူညီလဲ/အကောင်းဆုံးလဲ" ဆိုပြီး မဲခွဲဆုံးဖြတ် (Voting) တဲ့ စနစ် ဖြစ်ပါတယ်။

Agentic AI: Reflect Pattern

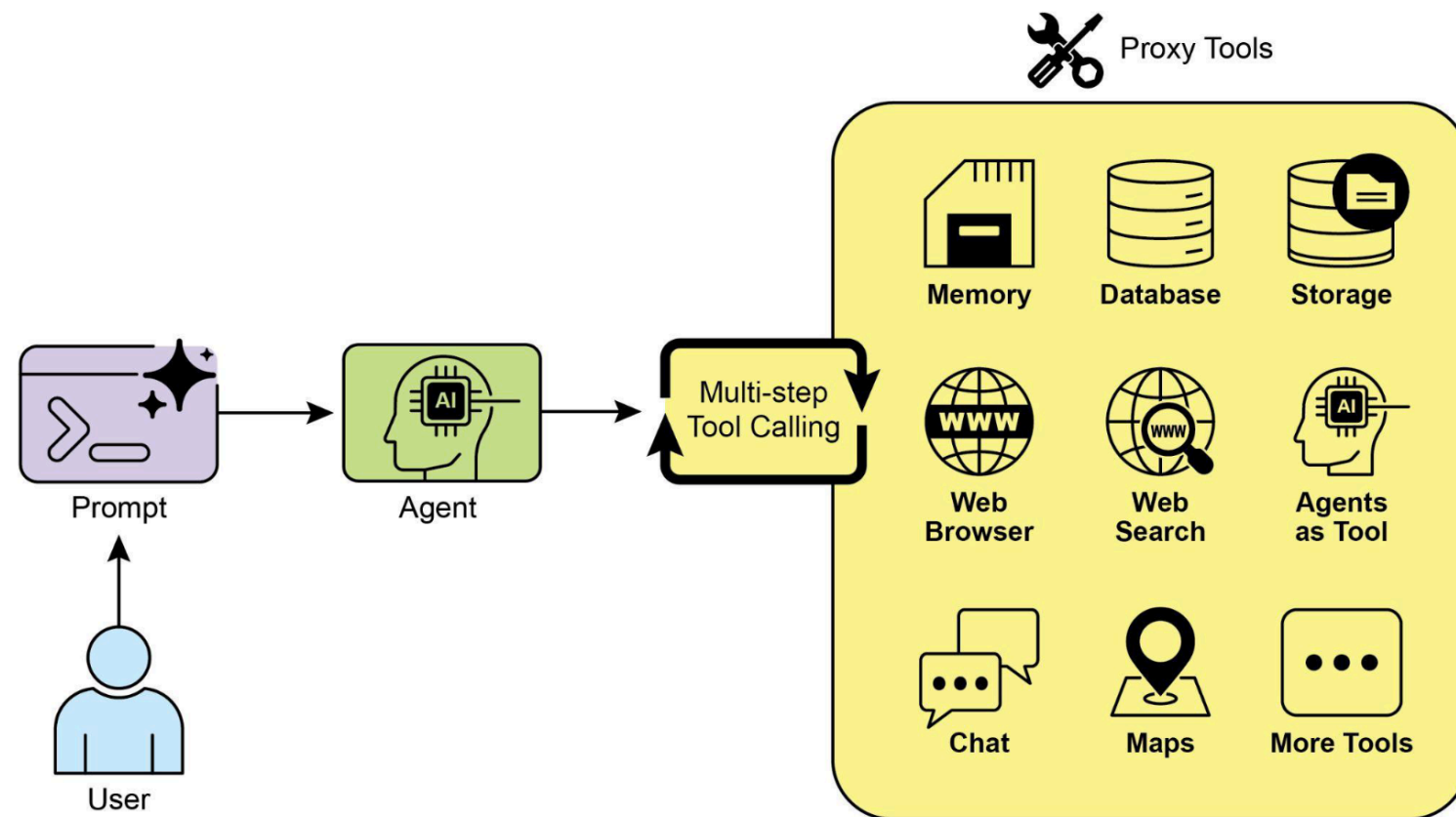


- **Single Agent Reflection (မိမိကိုယ်တိုင် ပြန်စစ်ခြင်း):** ဒီစနစ်မှာတော့ AI Agent တစ်ခုတည်းကပဲ အလုပ်လုပ်တာ ဖြစ်ပါတယ်။ ပေးလိုက်တဲ့ Prompt အပေါ်မူတည်ပြီး အဖြေထုတ်ပေးရုံတင် မဟုတ်ဘဲ၊ မိမိထုတ်ထားတဲ့ အဖြေကို ကိုယ်တိုင်အမှားရှာပြီး (Reflection Loop) အဆင့်ဆင့် ပြန်လည်ပြင်ဆင်ပြီးမှ စိတ်ချရတဲ့ Output ကို ယူဆာ (User) ဆီ ပြန်ပို့ပေးတဲ့ စနစ် ဖြစ်ပါတယ်။



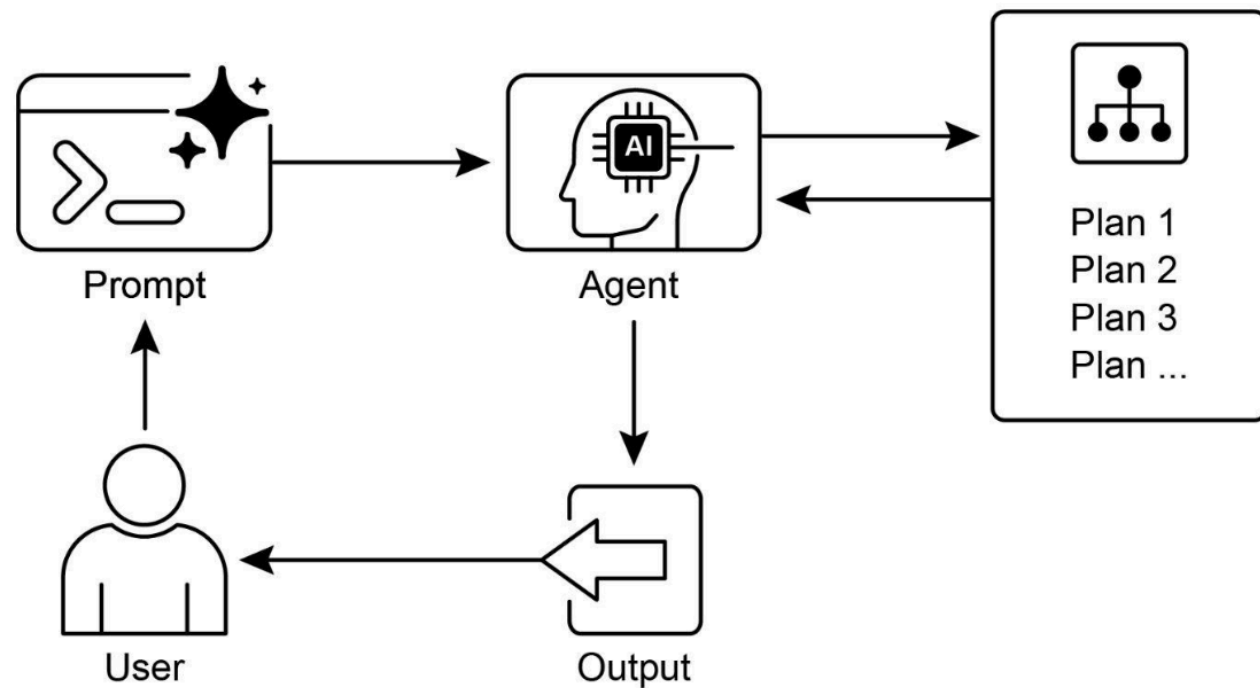
- **Producer-Critique Pattern (အဖွဲ့လိုက် အပြန်အလှန်စစ်ခြင်း):** ဒီစနစ်မှာတော့ သီးခြား Agent နှစ်ခု ခွဲသုံးထားပါတယ်။ Agent: Producer က ပထမဦးစွာ အဖြေကို ဖန်တီးပေးပြီး၊ Agent: Critique က ထိုအဖြေရဲ့ အားနည်းချက်တွေကို လိုက်လံဝေဖန် ထောက်ပြပါတယ်။ ထိုဝေဖန်ချက် (Reflection) ကို Producer ထံ ပြန်ပို့ပေးပြီး အကောင်းဆုံးဖြစ်အောင် ထပ်ခါထပ်ခါ ပြန်ပြင်ခိုင်းပြီးမှ Output ထုတ်ပေးတဲ့ စနစ် ဖြစ်ပါတယ်။

Agentic AI: Function Calling Pattern



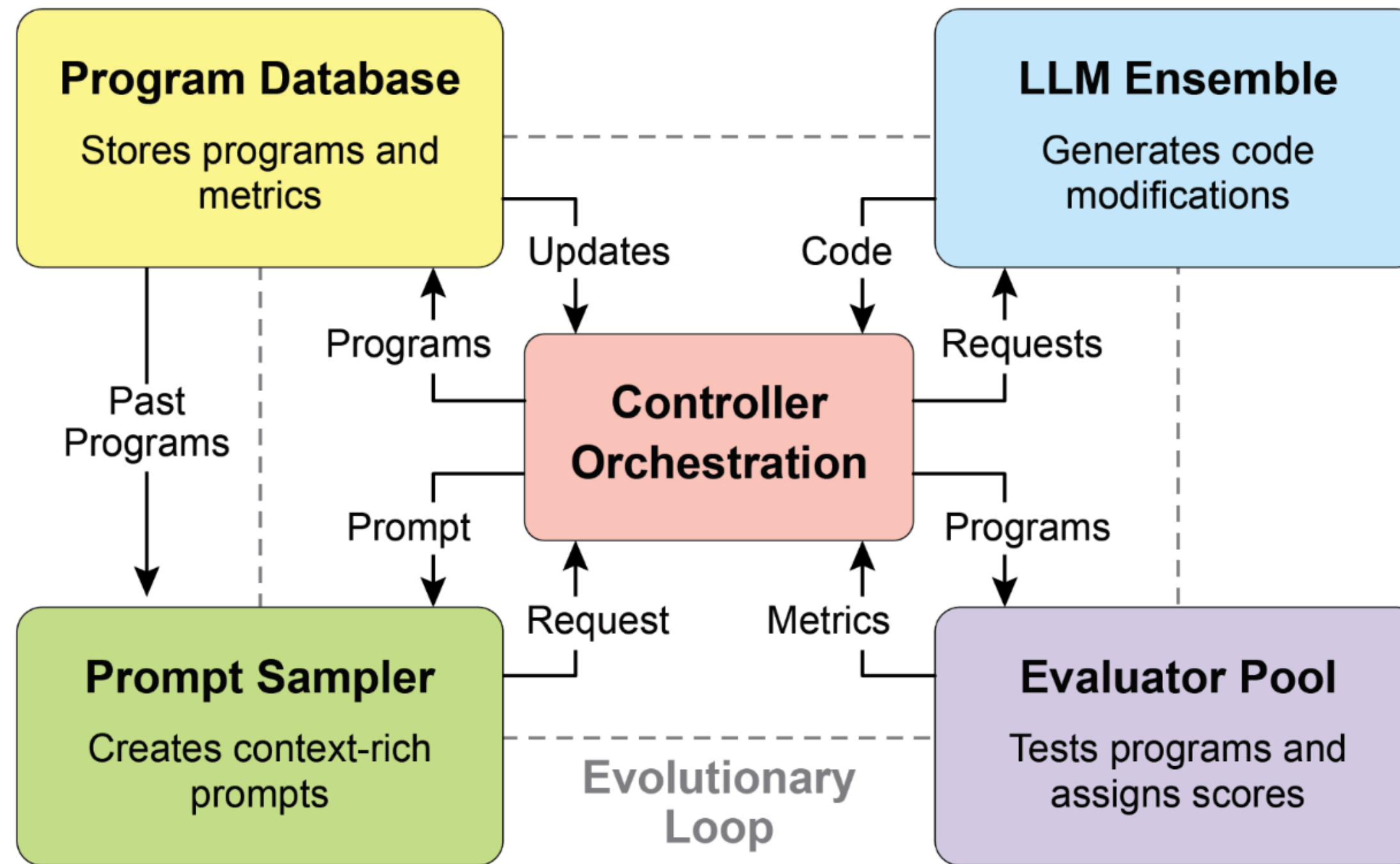
- **Multi-step Tool Calling:** AI Agent က ယူဆာ (User) ရဲ့ Prompt ကို လက်ခံရရှိတဲ့အခါ အဖြေကို တိုက်ရိုက်မခန့်မှန်းဘဲ မိမိမှာရှိတဲ့ ပြင်ပ Tools တွေကို အဆင့်ဆင့် ခေါ်ယူအသုံးပြု (Multi-step Calling) တဲ့ စနစ် ဖြစ်ပါတယ်။ အလုပ်တစ်ခု ပြီးမြောက်ဖို့အတွက် ဘယ် Tool ကို ဘယ်အချိန်မှာ သုံးရမလဲဆိုတာကို Agent က ကိုယ်တိုင် ဆုံးဖြတ်ပါတယ်။
- **Proxy Tools Ecosystem:** AI Agent က ကူညီလုပ်ဆောင်ပေးနိုင်တဲ့ ပြင်ပစနစ် (Ecosystem) တွေ ဖြစ်ပါတယ်။ ၎င်းတွင် မှတ်ဉာဏ် သိမ်းဆည်းခြင်း (Memory/Database)၊ အင်တာနက် ရှာဖွေခြင်း (Web Search/Browser)၊ မြေပုံကြည့်ခြင်း (Maps) နှင့် အခြားသော AI Agents များကိုပါ Tool တစ်ခုအနေဖြင့် ပြန်လည်ခေါ်ယူ သုံးစွဲခြင်း (Agents as Tool) တို့ ပါဝင်ပါတယ်။

Agentic AI: Planning Pattern



- **Goal Decomposition (အလုပ်ကို အဆင့်ဆင့်ခွဲခြမ်းခြင်း):** AI Agent က User ဆီက ရှုပ်ထွေးတဲ့ Prompt ဒါမှမဟုတ် ပန်းတိုင် (Goal) တစ်ခုကို လက်ခံရရှိတဲ့အခါ ချက်ချင်း အဖြေမထုတ်သေးဘဲ ပြီးမြောက်အောင် လုပ်ဆောင်ရမယ့် အစီအစဉ်တွေကို ကြိုတင်တွက်ချက်ပါတယ်။ ကြီးမားတဲ့ အလုပ်ကြီးကို Plan 1, Plan 2, Plan 3 ဆိုပြီး လုပ်ဆောင်ရမယ့် Tasks လေးတွေအဖြစ် စနစ်တကျ အဆင့်ဆင့် ခွဲခြမ်းစိတ်ဖြာလိုက် တဲ့ ပုံစံ ဖြစ်ပါတယ်။
- **Dynamic Execution Loop (အခြေအနေပေါ်မူတည်၍ ပြောင်းလဲလုပ်ဆောင်ခြင်း):** Agent က မိမိချမှတ်ထားတဲ့ Plan အတိုင်း တစ်ဆင့်ချင်းစီကို လိုက်နာလုပ်ဆောင်ပါ တယ်။ အလုပ်လုပ်နေစဉ်အတွင်း အခက်အခဲ (သို့) ရလဒ်အသစ်များအပေါ်မူတည်ပြီး မိမိရဲ့ နောက်ထပ် Plan တွေကို လိုအပ်သလို အချိန်နဲ့တပြေးညီ လိုက်လံမွမ်းမံပြင်ဆင် (Dynamic Adjust) နိုင်စွမ်းရှိပြီး၊ အစီအစဉ်အားလုံး ပြီးမြောက်မှသာ ယူဆာထံသို့ စိတ်ချရတဲ့ Output ကို ထုတ်ပေးတဲ့ စနစ် ဖြစ်ပါတယ်။

OpenEvolve Architecture



Asynchronous Pipeline Optimized for Maximum Throughput



OpenEvolve: Teaching LLMs to Discover Algorithms Through Evolution - Algorithmic SuperIntelligence...
Algorithmic SuperIntelligence Labs - AI that invents the next generation of algorithms. We deploy AI-discovered...
algorithmicsuperintelligence.ai

OpenEvolve Architecture

- **Program Sampler:** စနစ်ထဲမှာ ရှိနှင့်ပြီးသား ကုဒ်ဟောင်းတွေထဲကနေ အလားအလာအကောင်းဆုံး ကုဒ်တွေကို ရွေးချယ်ပေးတဲ့အပိုင်း ဖြစ်ပါတယ်။ ရွေးချယ်ထားတဲ့ ကုဒ်တွေရဲ့ အချက်အလက်တွေနဲ့ သမိုင်းကြောင်းတွေကို စုစည်းပြီး LLM ဆီ ပို့ပေးဖို့ Prompt အဖြစ် ပြင်ဆင်ပေးပါတယ်။
- **LLM Ensemble:** ကုဒ်တွေကို လိုက်လံပြုပြင်ပြောင်းလဲပေးတဲ့ အဓိက အင်ဂျင် (Mutation Engine) ဖြစ်ပါတယ်။ မတူညီတဲ့ AI Models အပြားကို စုပေါင်းအသုံးပြုပြီး လက်ရှိကုဒ်တွေကို ပုံစံအမျိုးမျိုး ပြောင်းလဲရေးသားခြင်း သို့မဟုတ် Algorithm အသစ်များ ဖန်တီးခြင်းတို့ကို လုပ်ဆောင်ပါတယ်။
- **Evaluator Pool:** ထွက်လာတဲ့ ကုဒ်အသစ်တွေကို တပြိုင်တည်း (In Parallel) စမ်းသပ်မောင်းနှင် (Test) ပေးတဲ့ အပိုင်းဖြစ်ပါတယ်။ ကုဒ်တစ်ခုချင်းစီရဲ့ အလုပ်လုပ်ပုံ မြန်နှုန်း၊ စွမ်းအင်သုံးစွဲမှုနဲ့ တိကျမှုတွေအပေါ်မူတည်ပြီး ကြံ့ခိုင်မှုရမှတ် (Fitness Score) တွေကို တိတိကျကျ သတ်မှတ်ပေးပါတယ်။
- **Program Database:** စမ်းသပ်ပြီးသမျှ ကုဒ်ဗားရှင်းအားလုံးနဲ့ ၎င်းတို့ရဲ့ ရမှတ်တွေကို စနစ်တကျ သိမ်းဆည်းထားတဲ့ နေရာဖြစ်ပါတယ်။ အဖြေတွေကို အသွင်သဏ္ဌာန်အစုံအလင် ရရှိစေဖို့အတွက် Quality-Diversity (MAP-Elites) စနစ်ကို အသုံးပြုပြီး ထိန်းသိမ်းပါတယ်။

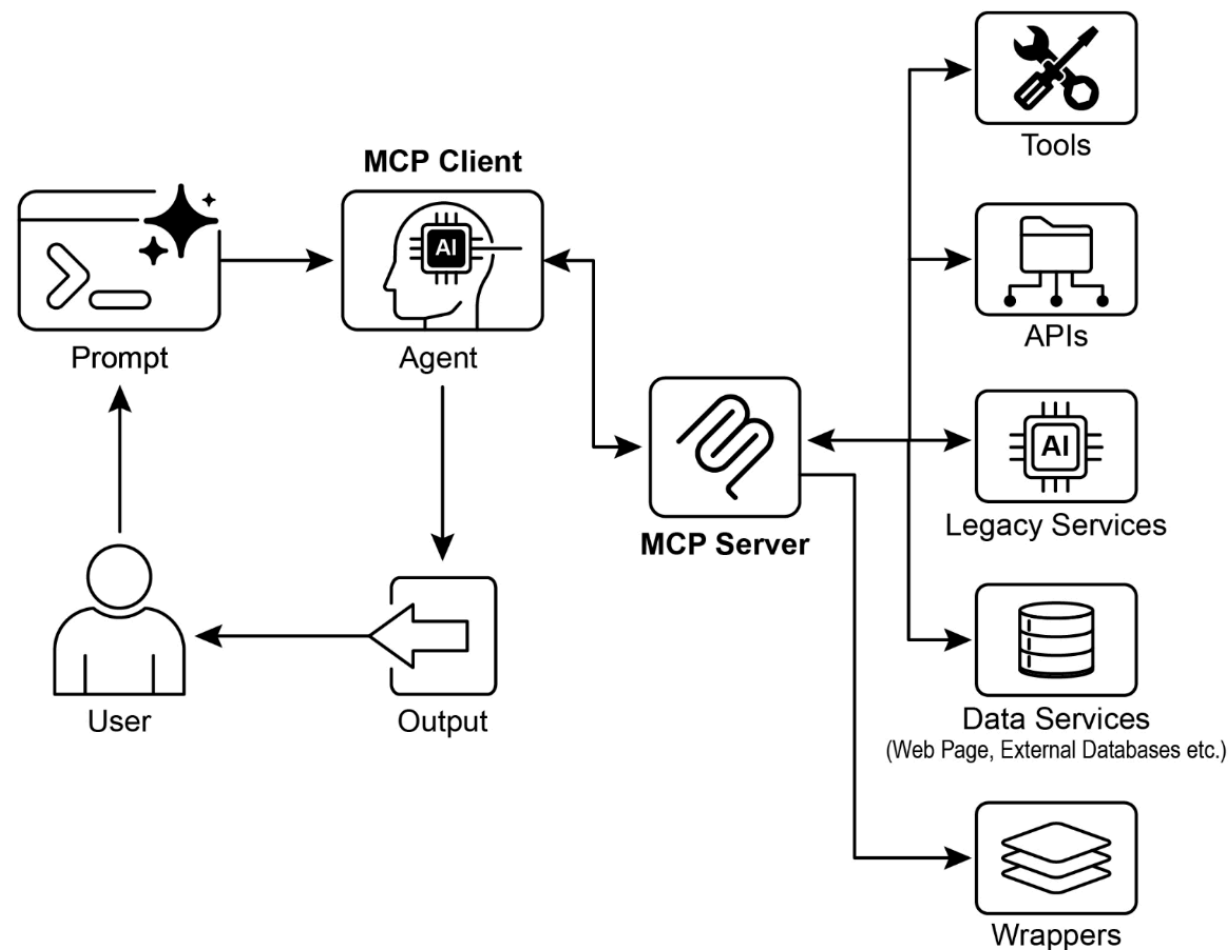
OpenEvolve Architecture (Cont'd)

- **Island-Based Topology:** ကုဒ်တွေကို အုပ်စုအသီးသီး (Islands) ခွဲပြီး တစ်ပြိုင်တည်း သီးခြားစီ တိုးတက်ပြောင်းလဲစေတဲ့ စနစ်ဖြစ်ပါတယ်။ ၎င်းက ကုဒ်တွေအားလုံး ပုံစံတစ်သမတ်တည်းဖြစ်ပြီး လမ်းဆုံးသွားခြင်းမျိုးမရှိအောင် ကာကွယ်ပေးပါတယ်။
- **Artifact Side-Channel:** ကုဒ်တွေ မောင်းနှင်ရာမှာ အမှား (Errors) တက်ခဲ့ရင် အဲဒီအမှားတွေကို ချန်မထားဘဲ လာမယ့် Prompt ထဲကို တိုက်ရိုက်ပြန်ထည့်ပြီး AI ကို အမှားပြင်ခိုင်းတဲ့ (Self-Correction) လမ်းကြောင်း ဖြစ်ပါတယ်။
- **Double Selection Strategy:** ကုဒ်တွေကို ရွေးချယ်ရာမှာ စံနှုန်းနှစ်ခု သုံးပါတယ်။ တစ်ခုက လက်ရှိစွမ်းဆောင်ရည် အကောင်းဆုံး ကုဒ်တွေကို ရွေးချယ်ခြင်းဖြစ်ပြီး၊ အခြားတစ်ခုကတော့ နောက်ထပ် စိတ်ကူးသစ်တွေရနိုင်မယ့် တည်ဆောက်ပုံဆန်းသစ်တဲ့ ကုဒ်တွေကို ရွေးချယ်ခြင်း ဖြစ်ပါတယ်။
- **Deterministic Seeding:** လုပ်ဆောင်ချက်တိုင်းကို ကိန်းဂဏန်းစနစ် (Global Seed) တစ်ခုတည်းအောက်မှာ ထိန်းချုပ်ထားတဲ့ အတွက် ဘယ်နှစ်ကြိမ်ပဲ ပြန်လည်စမ်းသပ် စမ်းသပ်၊ တူညီတဲ့ ရလဒ်ကောင်းကို ပုံသေ ပြန်လည်ရရှိစေပါတယ်။

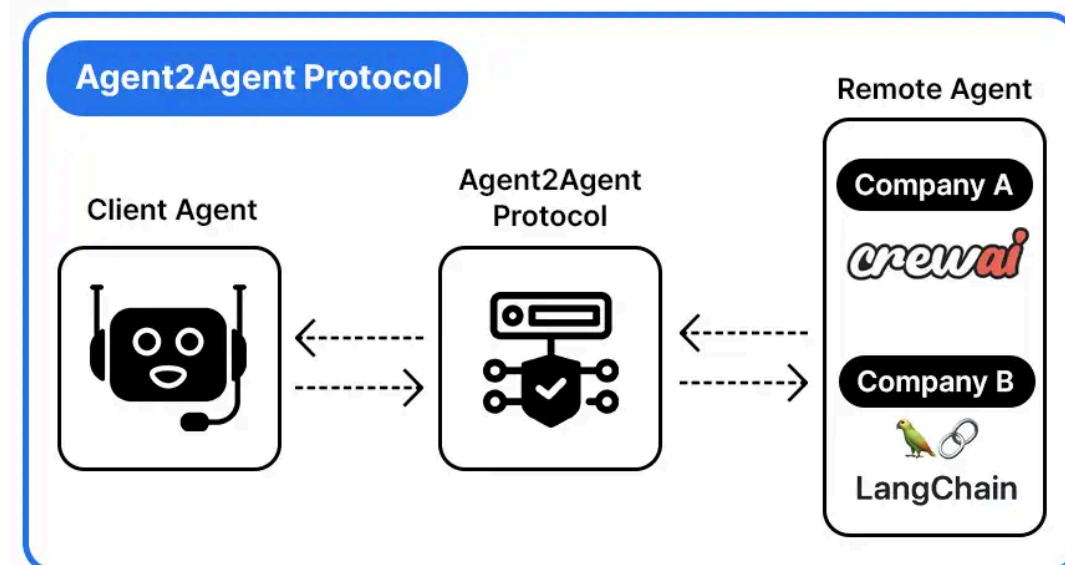
OpenEvolve Architecture (Cont'd)

- Parallel Pattern (တစ်ပြိုင်တည်း ခွဲလုပ်ခြင်းစနစ် အသုံးချပုံ):
 - Evaluator Pool: OpenEvolve သည် ထွက်လာသော ကုဒ်ဗားရှင်း အမြောက်အမြားကို တစ်ခုချင်းစီ စောင့်မနေဘဲ Evaluator Pool ထဲတွင် တစ်ပြိုင်တည်း (In Parallel) မောင်းနှင်စမ်းသပ်ပါသည်။
 - Island-Based Topology: ကုဒ်မျိုးရိုးဗီဇ (Populations) များကို အုပ်စုအသီးသီး (Islands) ခွဲထုတ်ပြီး သီးခြားစီ တစ်ပြိုင်တည်း တိုးတက်ပြောင်းလဲစေခြင်းဖြင့် အလုပ်လုပ်ချိန်ကို များစွာ လျော့ချပေးပါသည်။
- Reflection Pattern (ပြန်လည်ဆန်းစစ် ပြင်ဆင်ခြင်းစနစ် အသုံးချပုံ):
 - Artifact Side-Channel (Critic Loop): စမ်းသပ်မှုတွင် ကျရှုံးသွားသော ကုဒ်များ၏ Runtime Errors နှင့် Execution Traces (အမှားမှတ်တမ်း) များကို သီးခြားလမ်းကြောင်းဖြင့် ပြန်လည်ထုတ်ယူပါသည်။
 - Self-Correction: အဆိုပါ အမှားမှတ်တမ်းများကို နောက်တစ်ကြိမ် Prompt တွင်းသို့ ပြန်လည်ထည့်သွင်း (Inject) ပေးခြင်းဖြင့် LLM Ensemble အား အမှားကို သိရှိပြီး ပိုကောင်းအောင် ပြန်လည်ပြုပြင်ရေးသားစေသည့် Producer-Critique Dynamic ကို ဖန်တီးပါသည်။

Agentic AI: Model Context Protocol and A2A

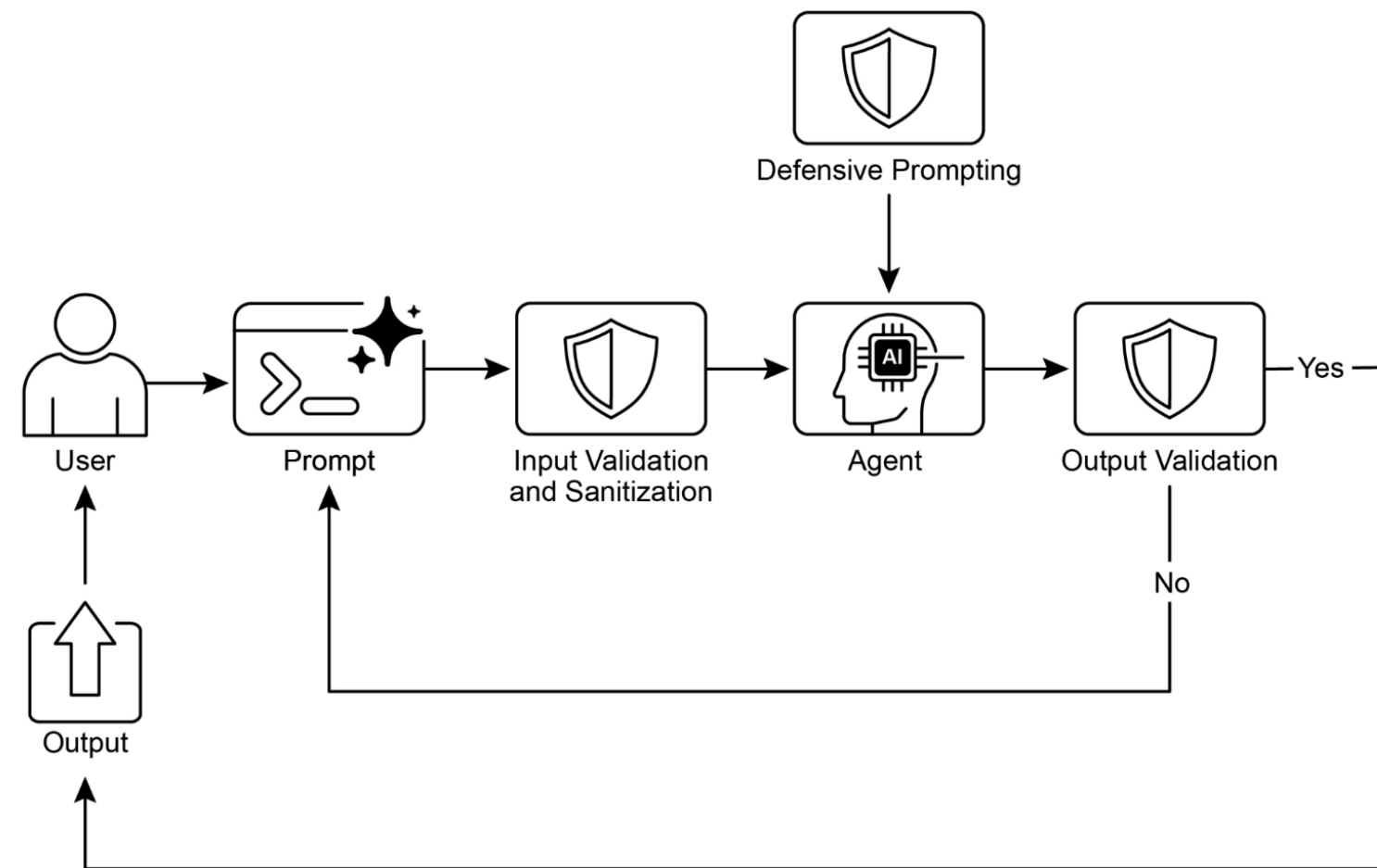


- **MCP Client (Agent):** User ထံမှ Prompt ကို လက်ခံပြီး၊ လိုအပ်သော ဒေတာနှင့် Tools များကို MCP Server ထံသို့ လှမ်းတောင်းပေးသည့် AI Agent ဖြစ်သည်။
- **MCP Server (ပေါင်းကူးတံတား):** AI Agent (Client) နှင့် ပြင်ပစနစ်များအကြား လုံခြုံစွာ ချိတ်ဆက်ပေးပြီး လမ်းကြောင်းလွှဲ (Route) ပေးသော ကြားခံဗဟိုချက် ဖြစ်သည်။
- **Connected Ecosystem (ပြင်ပစနစ်များ):** AI Agent အသုံးပြုနိုင်မည့် ပြင်ပအရင်းအမြစ်များ ဖြစ်ပြီး၊ ၎င်းတွင် Tools, APIs, Legacy/Data Services နှင့် Wrappers များ ပါဝင်သည်။



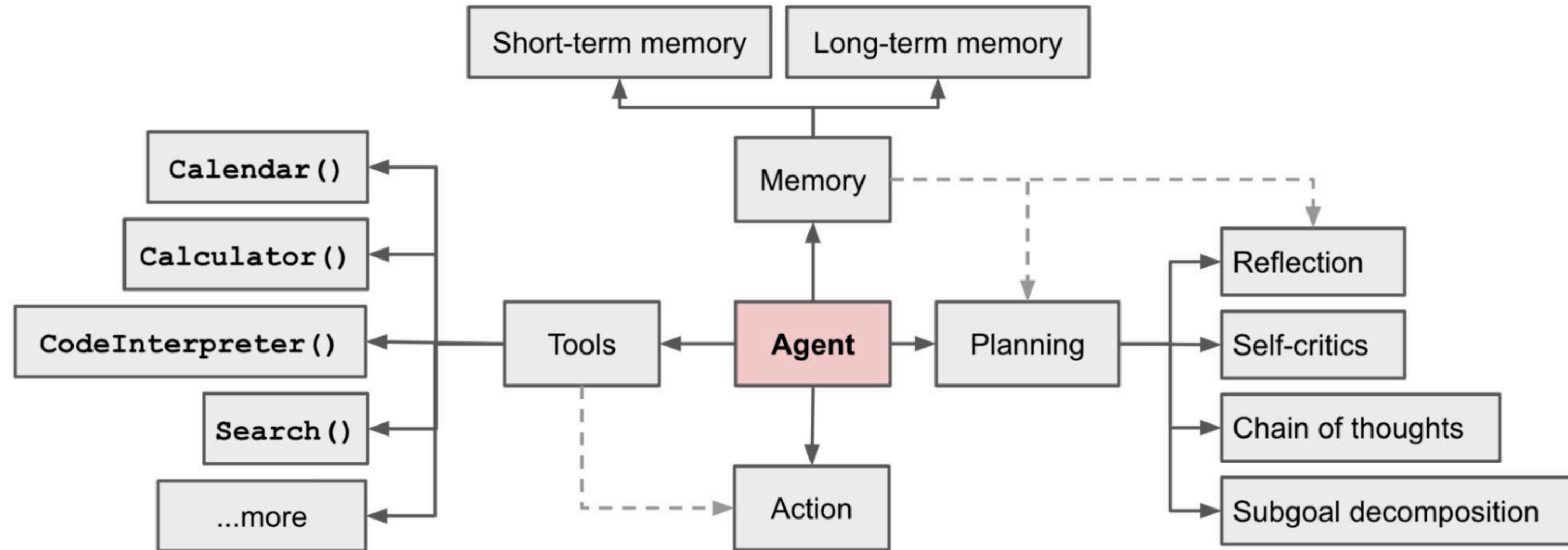
- Tool Use Pattern တွင် AI Agent သည် ပြင်ပ Tools များကို အဆင့်ဆင့် ခေါ်ယူအသုံးပြုသည်။
- MCP သည် ထိုသို့ ခေါ်ယူရာတွင် Tool တစ်ခုချင်းစီအတွက် ကုဒ်အသစ်များ ထပ်ခါထပ်ခါ ရေးစရာမလိုဘဲ Standard ပုံသေတစ်ခုတည်းဖြင့် အလွယ်တကူ ချိတ်ဆက်ခေါ်ယူနိုင်အောင် ဗဟိုမှ ပရိုတိုကော စံနှုန်း သတ်မှတ်ပေးခြင်း ဖြစ်သည်။
- AI Agent အနေဖြင့် Tool အသစ်များကို တိုက်ရိုက် လိုက်လံချိတ်ဆက်နေစရာ မလိုတော့ဘဲ MCP Server တစ်ခုတည်းနှင့် ချိတ်ဆက်လိုက်ရုံဖြင့် ၎င်းနောက်ကွယ်ရှိ Tools, APIs နှင့် Databases အားလုံးကို Tool Use Pattern အတိုင်း တစ်ပါတည်း တိုးချဲ့အသုံးပြုနိုင်သွားခြင်း ဖြစ်သည်။

Agentic AI: Guardrail Design Pattern



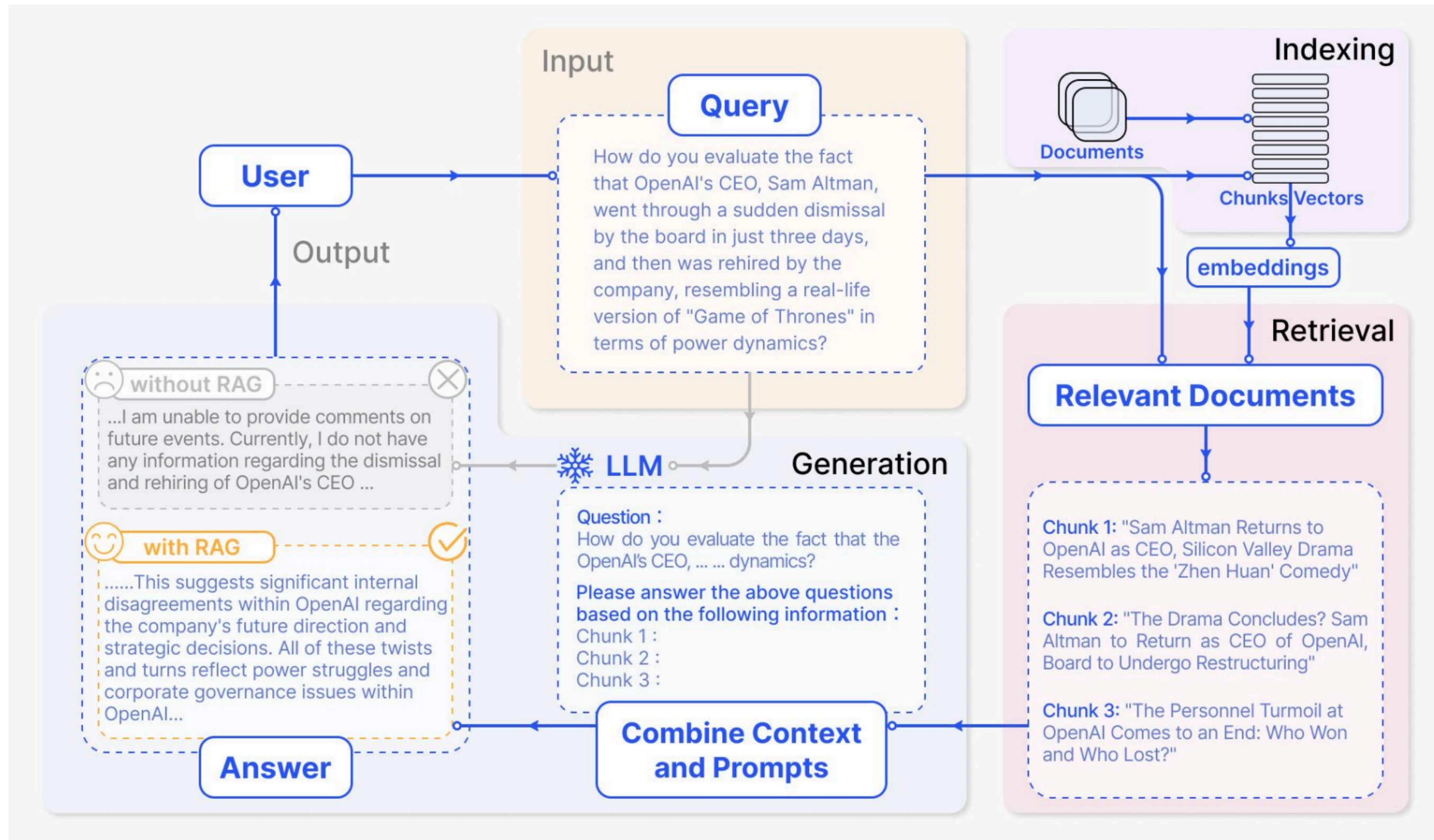
- Input Validation: Prompt ထဲက အန္တရာယ်ရှိတဲ့ ညွှန်ကြားချက်တွေကို အရင်ဖြတ်စစ်တယ်။
- Defensive Prompting: စည်းကမ်းမချိုးဖောက်ဖို့နဲ့ မလိမ်ညာဖို့ AI ကို စနစ်ထဲကနေ ကြိုပိတ်ထားတယ်။
- Output Validation Loop: အဖြေကို ထပ်စစ်ပြီး အဆင်ပြေရင် (Yes) ထုတ်ပြတယ်။ မပြည့်စုံရင် (No) အစကနေ ပြန်ပြင်ခိုင်းတယ်။

Structure of Agent

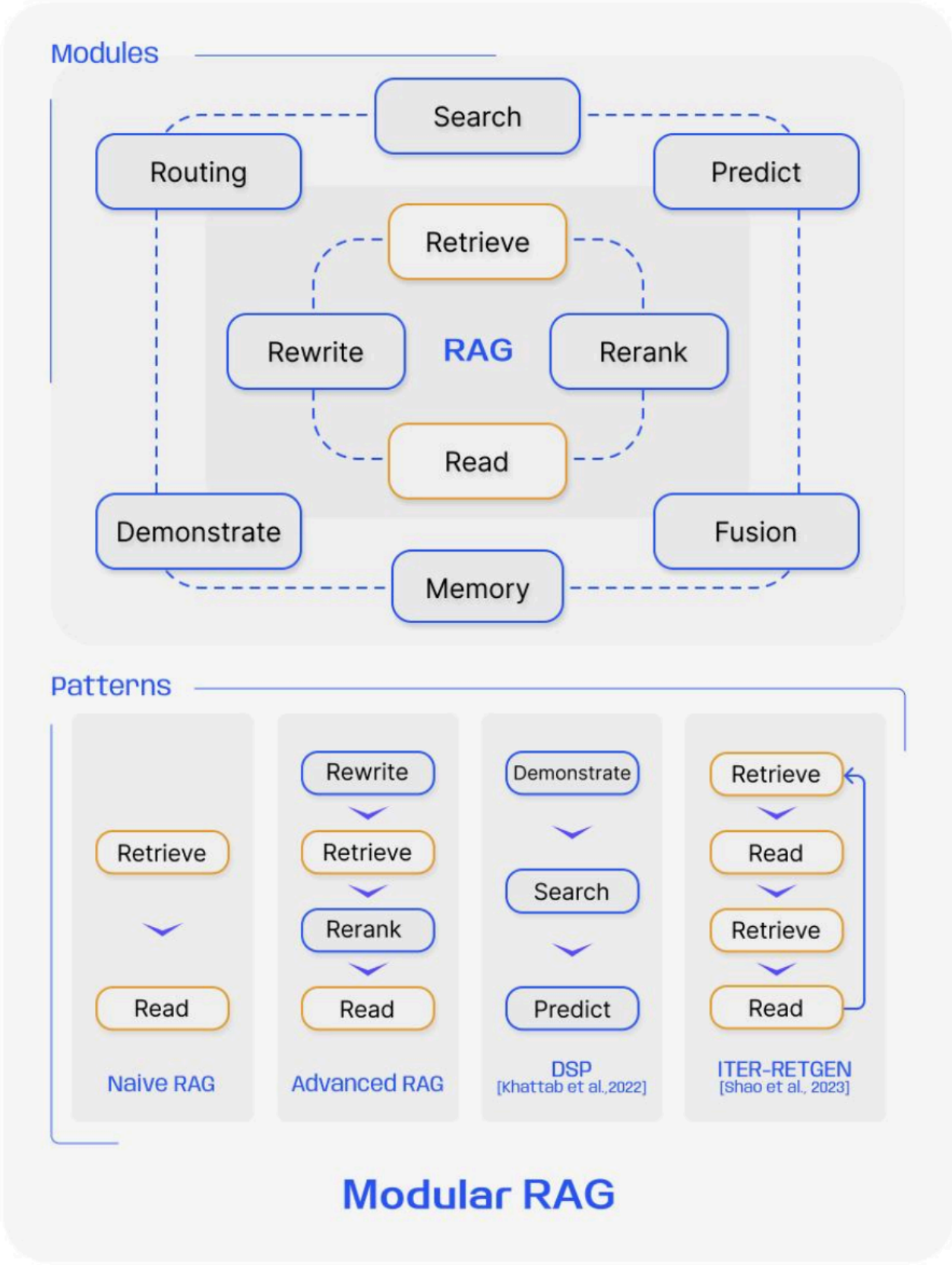
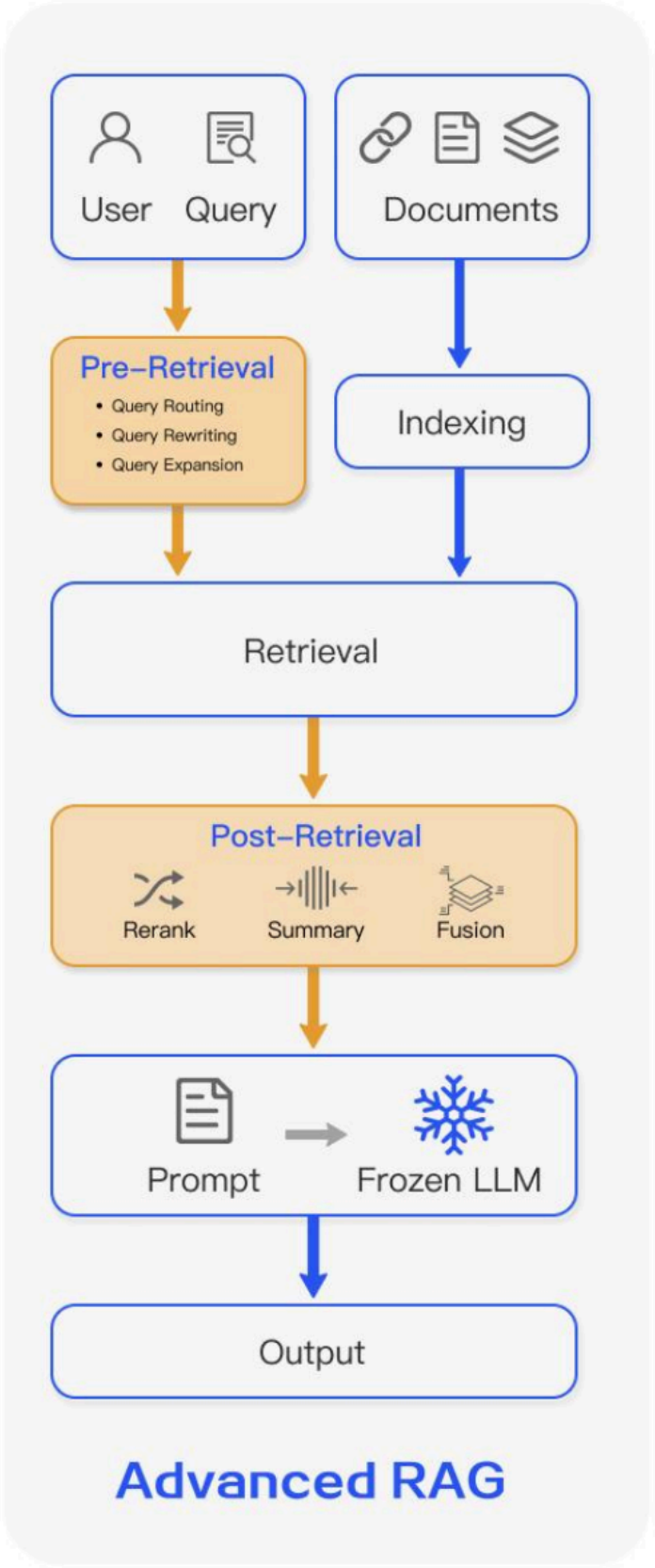
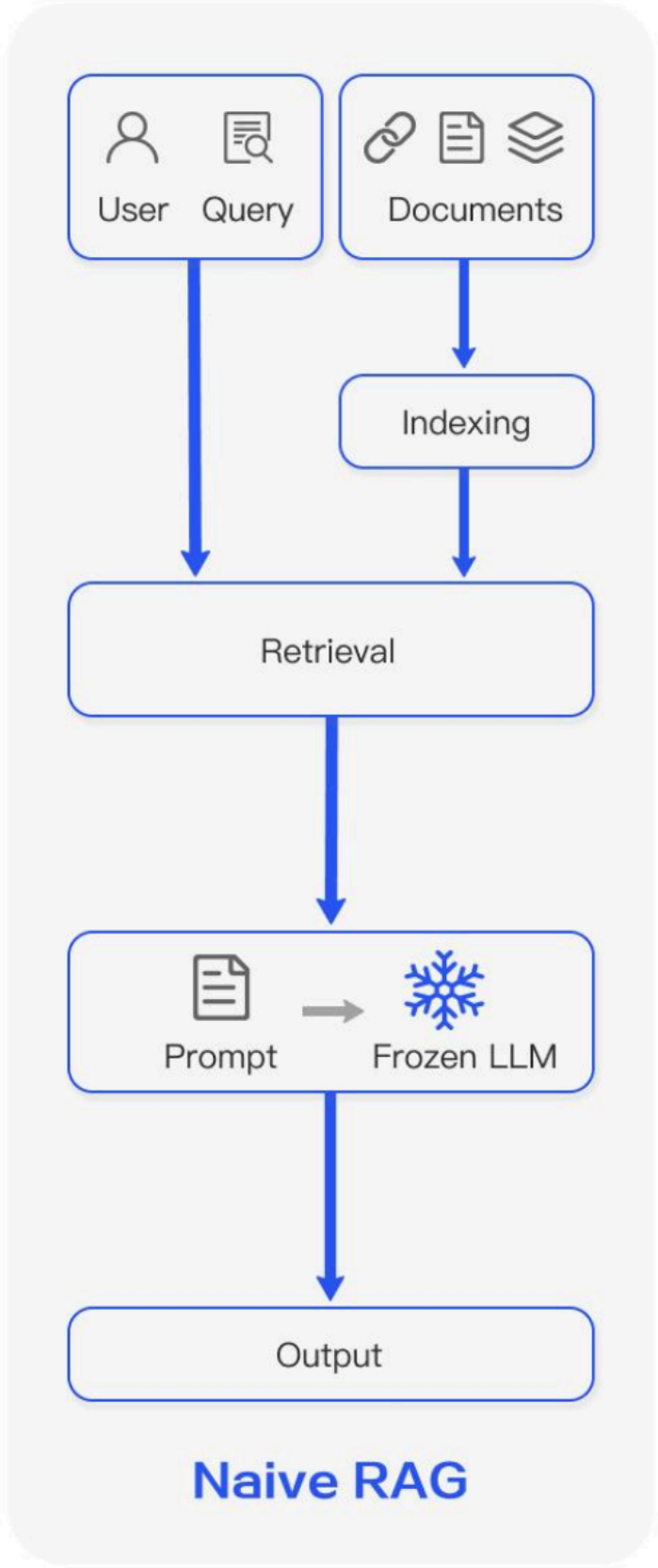


Source: <https://lilianweng.github.io/posts/2023-06-23-agent/>

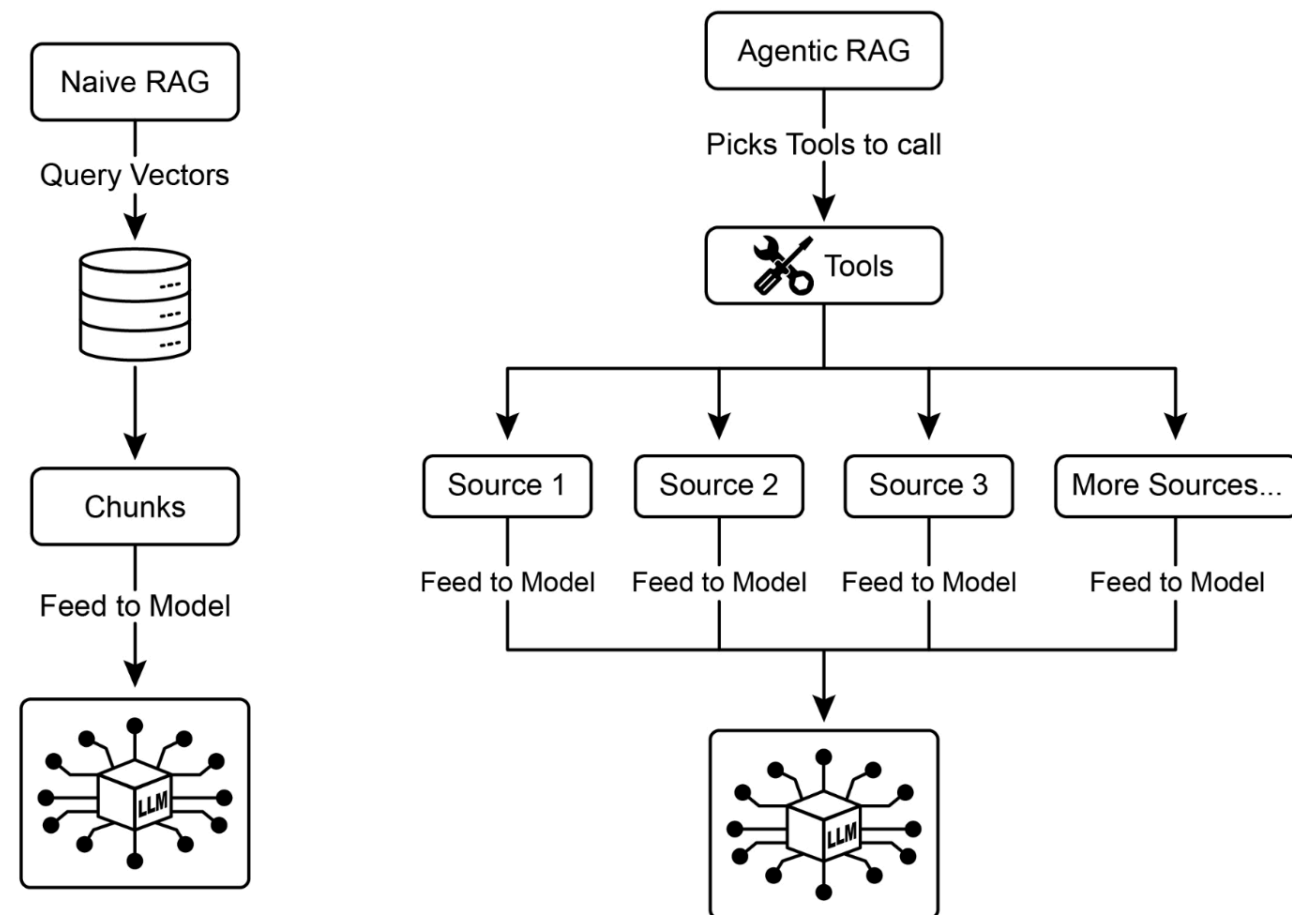
RAG



RAG (Cont'd)

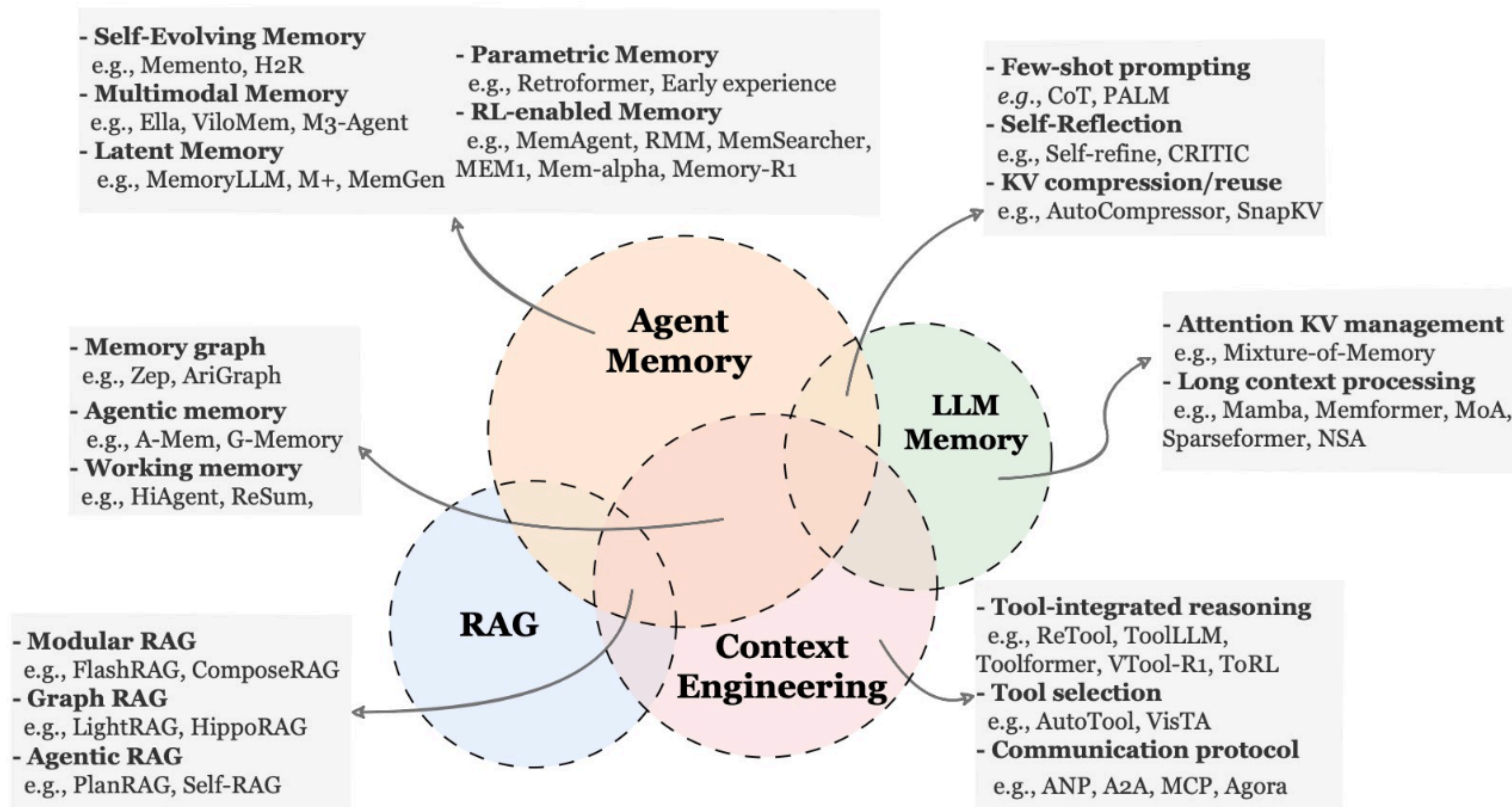


Agentic RAG



- အဆင့်ဆင့် လုပ်ဆောင်ပုံ: Agentic RAG က ဝင်လာတဲ့ မေးခွန်းကို ကြည့်ပြီး ဘယ် Tool ကို သုံးရမလဲဆိုတာ အရင်ဆုံး ရွေးချယ်တယ် (Picks Tools to call)။
- ပိုပြီး စမတ်ကျတဲ့ အချက်: ရွေးချယ်လိုက်တဲ့ Tools တွေကတစ်ဆင့် Source 1, Source 2, Source 3 စသဖြင့် သက်ဆိုင်ရာ Data အရင်းအမြစ်မျိုးစုံဆီကနေ ခွဲပြီး ရှာဖွေပေးတယ်။
- နောက်ဆုံးအဆင့်: အဲဒီမတူညီတဲ့ အရင်းအမြစ်တွေက ရလာတဲ့ Context အချက်အလက်အားလုံးကို ပေါင်းစပ်ပြီးမှ LLM ထံ ပေးပို့ (Feed to Model) ပြီး အဖြေထုတ်ခိုင်းတာ ဖြစ်ပါတယ်။

AI Agent Memory, RAG, and Context Engineering



Source: Hu, Y., Liu, S., Yue, Y., Zhang, G., Liu, B., Zhu, F., ... & Yan, S. (2025). Memory in the age of ai agents. arXiv preprint arXiv:2512.13564.

Materials for further study

- EMNLP 2024 Tutorial: Language Agents: Foundations, Prospects, and Risks, Link: <https://language-agent-tutorial.github.io/>
- Agentic Design Patterns, Antonio Gulli (2025)
- CS294/194-196 Large Language Model Agents, 2024, UC Berkley. Link: <https://rdi.berkeley.edu/llm-agents/f24>
- ADK Crash Course, Google, Link: <https://codelabs.developers.google.com/onramp/instructions#0>
- Agentic Memory: Learning Unified Long-Term and Short-Term Memory Management for Large Language Model Agents, Yi Yu et. al. (2026) Link: <https://arxiv.org/pdf/2601.01885>

Thank you.

Questions are welcomed.